FRIEDRICH-ALEXANDER-UNIVERSITÄT ERLANGEN-NÜRNBERG
TECHNISCHE FAKULTÄT • DEPARTMENT INFORMATIK

Lehrstuhl für Informatik 10 (Systemsimulation)

FRAUNHOFER-INSTITUT FÜR KURZZEITDYNAMIK
ERNST-MACH-INSTITUT • EMI

COMPRESSIBLE LATTICE BOLTZMANN SOLVER FOR SUPERSONIC FLOW

Sudesh Rathnayake

Master Thesis

COMPRESSIBLE LATTICE BOLTZMANN SOLVER FOR SUPERSONIC FLOW

Sudesh Rathnayake

Master Thesis

Aufgabensteller:	Prof.Dr.-Ing. Harald Köstler
Aufgabensteller:	Maximilian Blechner, M.Sc.
Betreuer:	Prof.Dr.-Ing. Harald Köstler
Betreuer:	Maximilian Blechner, M.Sc.
Bearbeitungszeitraum:	22.09.2023 – 22.03.2024

Erklärung:

Ich versichere, dass ich die Arbeit ohne fremde Hilfe und ohne Benutzung anderer als der angegebenen Quellen angefertigt habe und dass die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen hat und von dieser als Teil einer Prüfungsleistung angenommen wurde. Alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, sind als solche gekennzeichnet.

Der Universität Erlangen-Nürnberg, vertreten durch den Lehrstuhl für Systemsimulation (Informatik 10), wird für Zwecke der Forschung und Lehre ein einfaches, kostenloses, zeitlich und örtlich unbeschränktes Nutzungsrecht an den Arbeitsergebnissen der Master Thesis einschließlich etwaiger Schutzrechte und Urheberrechte eingeräumt.

Erlangen, den 4. März 2024 .....

Acknowledgement

Despite the fact that a rough sea helps to make a good sailor, it is essential to have a proper compass that will guide the voyage to the intended destination. Thus, first of all, I am deeply grateful to my thesis supervisors, Prof. Dr. Harald Köstler and Mr. Maximilian Blechner, for their invaluable guidance, support, and encouragement throughout the entire process. Their expertise, patience, and insightful feedback have been instrumental in shaping this work and refining my research skills. I am also grateful to all my group members, especially to the group leader, Mr. Martin Hunzinger from the Department of Experimentelle Ballistik at the Ernst-Mach-Institut in Fraunhofer-Institut für Kurzzeitdynamik for providing me with the resources and facilities necessary for conducting this research. Furthermore, I would like to extend my special thanks to Dr. Christophe Coreixas from the University of Geneva, Dr. Si Bui Quang Tran from the Institute of High Performance Computing in Singapore, Mr. Samuel Kemmler, Mr. Dane Lacey and Mr. Nicolas Wilhelm for sharing some invaluable knowledge to shape up this work. I would be remiss not to mention my parents and my wife for their unwavering love, encouragement, and understanding during this challenging journey. Their endless support has been my source of strength and motivation. Thank you all for being a part of this journey.

Abstract

This thesis presents a compressible lattice Boltzmann solver to investigate supersonic flow phenomena. Building upon existing lattice Boltzmann techniques primarily used in incompressible flows, this research extends the method to the compressible regime using DDF-ELBM, offering a unique approach to simulating complex supersonic flows. Through careful algorithmic development and rigorous validation against analytical solutions and experimental data, the solver demonstrates its effectiveness in capturing key features of supersonic flow, including shock structures and expansion waves. Key findings include the solver's ability to accurately predict flow properties such as Mach number, pressure, temperature, and density distribution in various supersonic flow scenarios. Additionally, the solver's performance makes it a valuable tool for investigating compressible flow phenomena at high Reynolds numbers and high Mach numbers. Even though the concepts of DDF-ELBM are not entirely novel, this approach represents a significant advancement in the application of the Lattice Boltzmann Method (LBM) to the study of supersonic flows, offering insights into flow behavior and contributing to ongoing LBM software framework developments such as the widely applicable lattice Boltzmann simulation code from Erlangen (waLBerla).

Contents

List of Figures

List of Tables

List of Algorithms

Chapter 1

Introduction

1.1 Motivation

Computational Fluid Dynamics (CFD) has been an emerging field when it comes to the understanding of fluid flow phenomena in many aspects. Starting from simple fluid flows, it is being used to describe the complex flow problems associated with several industries such as the food industry, medical industry, aerodynamic and aerospace applications as well as in defence industry. However, it is still a challenging task when it comes to accurate predictions of Navier-Stokes Equations (NSE) efficiently even though with the advancement of modern-day supercomputers. Modern up-to-date CFD solvers widely use Finite Volume Method(FVM), Finite Element Method (FEM) or Finite Difference Method (FDM) techniques to predict the complex nature of flows accurately. In the recent past, the Lattice Boltzmann Method (LBM) has been introduced as an alternative technique to solve fluid flows and, it has shown its success in describing several flows, mainly in incompressible regimes. Further, due to the simplicity and the parallelizable nature of the LBM implementation, the method has become a popular topic among high performance computing community.
In most real-world applications, the nature of the flows is compressible. Therefore, when simulating compressible flows density variations, and temperature variations as well as the Mach numbers of the flow field should be properly captured to describe the flow field. Further, determining shock formations and discontinuities of the flow field due to shock formations are also some of the interesting as well as challenging tasks in compressible flow simulations. With the help of conventional CFD techniques these properties can predict accurately although it requires finer mesh resolutions. Eventually, these lead to higher computational time. Therefore, considering the simplicity and the efficiency of the LBM, it has been a dynamic research topic to adopt LBM to simulate compressible fluid flows. Conventionally, LBM is known as a "weakly compressible" solver for the NSE [1]. Hence, adopting the standard LBM scheme while preserving efficiency is not straightforward.
According to the author's knowledge, one of the first notable compressible flow simulations for high Mach number flows were proposed by Sun [2]. The proposed adaptive LB model consists of variable particle velocities based on the Mach number and internal energy. However, due to application inconvenience and numerical instability, there are few physical results based on these models. Kang et al. [3] discuss the simulation of shock wave fluid phenomenon using the Finite Difference Lattice Boltzmann Method (FDLBM) and propose a new model using the lattice BGK compressible fluid model in FDLBM to improve calculation speed and stabilize the numerical scheme. Tsutahara et al. [4] proposed an LB scheme coupled with a finite difference method for two-dimensional compressible flows. However, the model is expected to simulate compressible flows where the Mach number is less than one. Recently, Frapoli et al. [5] adopted a standard collide and stream LB scheme to simulate compressible flows at higher

Mach numbers considering entropic estimation of the equilibrium distribution. However, the model consists of non-standard higher-order lattice with $DdQ7^d$. According to Latt et al. [6], the model proposed by Frapolli [7] is by far the most accurate compressible LB scheme in the literature. Here, the model consists of two distribution functions, where one is for mass and momentum and, another one is considering energy. Latt et al. [6] proposed a compressible LB scheme using a double distribution approach where an accurate prediction of thermal compressible flows in a supersonic regime only considers 39 discrete velocities when compared to 343 discrete velocity model proposed by [7].
However, the usage of non-standard lattice models incorporates some complexities during implementation especially when it comes to the handling of boundary conditions. Further, higher-order lattices will lead to higher computational cost and memory usage. Therefore, the present focus and interest of the thesis are mainly in the domain of standard lattice models such as $DdQ3^d$. Saadat et al. [8] adopted above-mentioned double distribution approach to model compressible flows at high Mach numbers while using standard lattices. Further, an extension for the model considering gas dynamics was introduced [9] by the same group while using standard lattices to predict compressible flows. Very recently, Tran et al. [10] proposed compressible LBM using standard lattice and, according to the presented results, the model can simulate compressible flows up to Mach numbers 1.5. The proposed LB scheme in [10] will be also discussed in the following chapters.
To conclude, According to the present literature, it can be observed that compressible LBMs are a relatively novel and dynamic topic where investigations of the present models are still ongoing. The present thesis will mainly focus on simulating compressible flows greater than Mach number one using LBM while adopting standard lattice stencils. One of the main tasks of the thesis is to investigate the shock formation or shock capturing in compressible flows at higher Mach numbers efficiently. Further, the remaining sections of this chapter will elaborate on the problem description and the objectives of the thesis.

1.2 Problem Statement

In the realm of fluid dynamics, pressure disturbances manifest as propagating waves characterized by successive compressive and rarefaction phases, driven by the intrinsic elasticity of the fluid medium. This phenomenon finds an analogous counterpart in the transmission of sound waves through various mediums. The velocity at which a small pressure pulse or wave advances through a compressible medium is denominated as the speed of sound in that specific medium [11]. It is pivotal to emphasize that the speed of sound waves is intrinsically linked to the compressibility properties inherent to the medium through which they propagate, illustrating a fundamental relationship between wave behaviour and the compressive nature of the medium.
In the current endeavour, the primary focus is to investigate the feasibility of applying the LBM to simulate compressible fluid flows, rather than using conventional CFD techniques. By doing so, implementing a compressible lattice Boltzmann solver to investigate the propagation of pressure waves is mainly studied. Therefore, depicting the shock tube experiments using an accurate and computationally efficient lattice Boltzmann model is a proper starting point for investigating pressure wave propagation. This can be also considered as a Riemann problem which will be properly discussed in the following chapters.

1.3 Objectives

The primary objective of this master's thesis is to comprehensively investigate and implement a compressible fluid flow solver within the context of the LBM for flows where Mach number is greater than one. This research aims to achieve several interrelated goals:

1. **To Examine**: The first objective is to examine the limitations of the conventional LBM, in the context of compressible fluid simulations and investigate the behaviour of conventional LBM when there are discontinuities in the macroscopic properties in a domain of interest.

2. **To Investigate**: The second objective focuses on the investigation of the compressible LBM, aiming to simulate compressible fluid flows with Mach numbers greater than one. Further, the use of standard lattice stencils while using standard "collide" and "stream" algorithms is mainly considered.

3. **To Implement**: The third objective is to develop and implement compressible lattice Boltzmann algorithm. Here the algorithm is mainly implemented using C++ with OpenMP.

4. **To Evaluate**: The final objective involves the evaluation of the implented algorithm. This includes verification and validation of the implementation, performance evaluation of the present implementation and also to assess its practical applicability and effectiveness.

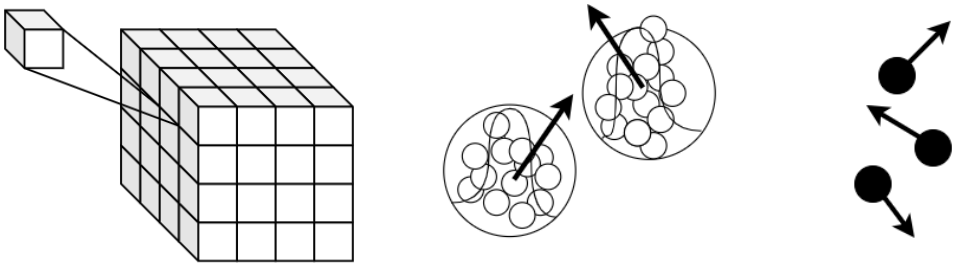
By accomplishing these objectives, this thesis seeks to contribute to the existing body of knowledge in the area of compressible LBM method by investigating and implementing a lattice Boltzmann scheme using a double distribution approach for higher Mach number flows. Moreover, the findings and recommendations derived from this research are expected to offer valuable guidance to existing lattice Boltzmann frameworks such as WaLBerla/lbmpy when adapting to compressible fluid flow simulations and serve as a foundation for further research and innovation in compressible LBM.

Chapter 2

Incompressible Lattice Boltzmann Method

2.1 Overview

As computational fluid dynamics becomes increasingly vital in modern engineering and scientific research, understanding the lattice Boltzmann method is essential. Its ability to handle complex geometries, parallelize simulations efficiently, and model multi-phase flows makes it a valuable tool in a wide range of applications.
Conventionally, continuum and discrete methods have been widely used to simulate mass, momentum and energy equations. For instance, in the continuum approach the above conservation equations apply to an infinitesimal control volume. Then, by using the FVM, FDM or FEM the differential equations are converted to a system of algebraic equations, of course, with the proper set of initial and boundary conditions. This scale is known as the macroscopic scale. In contrast, the other common approach is to consider the medium as a particulate flow. Here, Newton's second law of motion applied to each particle, and the properties of the medium are determined according to the behaviour of each particle. One example is molecular dynamic simulations, and this scale is known as the microscopic scale. However, in the LBM, the scale is mesoscopic, and it is in between the above-mentioned scales as shown in the following figure 2.1 [12]. In the mesoscopic scale, the particular property of one state is determined by using a



(a) Macroscopic scale (b) Mesoscopic scale (c) Microscopic scale

Figure 2.1: Simulation techniques in different scales

collection of particles. The main idea is that the particular collection of particles is bound to a distribution function which is mostly denoted as f in LBM literature. In LBM, the discretized Boltzmann equation determines the space and time evaluation of discrete-velocity distribution function $f(x,t)$. The discretised Boltzmann equation used in standard LBM can be expressed

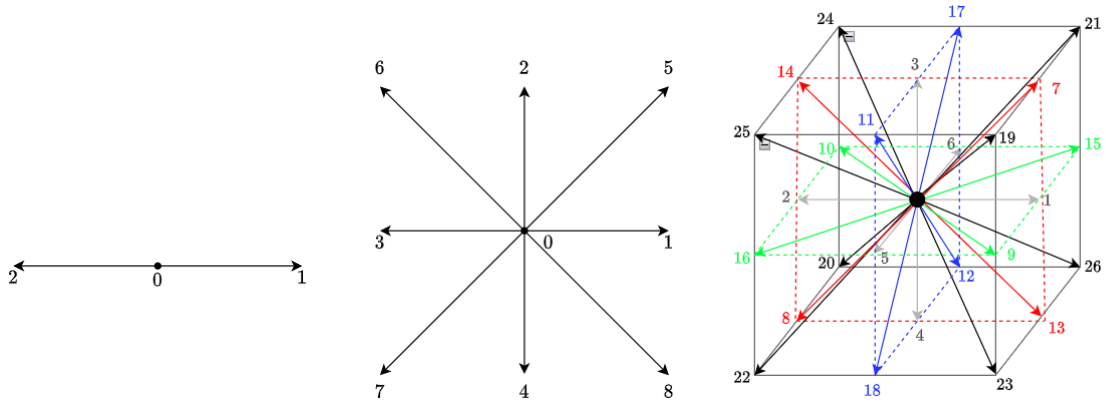
as follows.

$$f_i(x + c_i\delta t, t + \delta t) = f_i(x, t) + \Omega_i(x, t) \tag{2.1}$$

In equation 2.1, c_i represents the dimensionless discrete velocity, which dictates the movement of the respective distribution to the next lattice node based on the lattice arrangement. The collision operator, denoted as Ω_i maps the distributions to the same space while integrating the momentum exchanges resulting from particle collisions within a particular time step. A concise description of different collision operators will be described in a later topic in this chapter.

2.1.1 Lattice Stencils

In contrast to the mesh based discretisation schemes such as FVM, here in LBM, the domain of interest is discretised into a lattice. Each lattice node consists of a lattice stencil which is determined according to the requirements of dimensionality and required accuracy. One crucial aspect of LBM is that the calculations are carried out using lattice units, which are non-dimensional. Therefore, the connection between the physical model and the simulation is established using the law of similarity [1]. Considering this, one of the common space and time discretizations used in LBM is $\Delta x = \Delta y = \Delta z = \Delta t = 1$.
Here, within the scope of the master's thesis, the interested lattice stencils have the structure of $DdQ3^d$. D's represents the dimensionality, while the Q relates to the number of lattice velocities. Since these stencils are only interested in their immediate neighbourhood, this stencil category is commonly referred to as standard stencils. However, most of the previous studies in compressible LBM have adopted multi-speed stencils as mentioned in the previous chapter. The three main standard stencil configurations and their properties used in this work are shown in figure 2.2 and table 2.2 respectively [1].



(a) $D1Q3$ stencil (b) $D2Q9$ stencil (c) $D3Q27$ stencil

Figure 2.2: Standard stencils used in the current work

Stencil	c_i	p	w_i	c_s
$D1Q3$	0	1	2/3	$1/\sqrt{3}$
	± 1	2	1/6	
	(0,0)	1	4/9	
$D2Q9$	($\pm$1,0),(0,$\pm$1)	4	1/9	$1/\sqrt{3}$
	($\pm$1,$\pm$1)	4	1/36	
	(0,0,0)	1	8/27	
$D3Q27$	($\pm$1,0,0),(0,$\pm$1,0),(0,0,$\pm$1)	6	2/27	$1/\sqrt{3}$
	($\pm$1,$\pm$1,0),($\pm$1,0,$\pm$1),(0,$\pm$1,$\pm$1)	12	1/54	
	($\pm$1,$\pm$1,$\pm$1)	8	1/216	

Table 2.1: Properties of standard stencils used in the thesis scope

In table 2.2, c_i represents the abstract non-dimensional velocity vector representation in a stencil, where p denotes the number of c_i's of that kind. Further, w_i represents the weights associated with each c_i link, and c_s is the lattice speed of sound.

2.1.2 Streaming and Collision

The driving force of the LBM algorithm is the collision and streaming step. The discretised Boltzmann equation 2.1 combines the collision and streaming steps. To understand the collision step and the streaming step the equation 2.1 can be rewritten as follows,

$$\begin{aligned} &\textbf{collision} \quad : f^*{}_i(x,t) \qquad\qquad = f_i(x,t) + \Omega_i(x,t) \\ &\textbf{streaming} \quad : f_i(x + c_i\delta t, t + \delta t) = f^*{}_i(x,t) \end{aligned}$$

In the collision step, $f^*{}_i(x,t)$ represents the post-collision values of the discrete-velocity distribution function $f_i(x,t)$. Here, the discrete-velocity distributions transforms from $\mathbb{R}^D \rightarrow \mathbb{R}^D$. After collision, the post-collision values are propagated to the next lattice node according to the lattice stencil configurations. In LBM implementations, it is a common approach to allocate different arrays for pre-collision and post-collision discrete Boltzmann distributions. Based on this, there are two types of widely used streaming methods among the LBM community namely the "push" and "pull" methods.

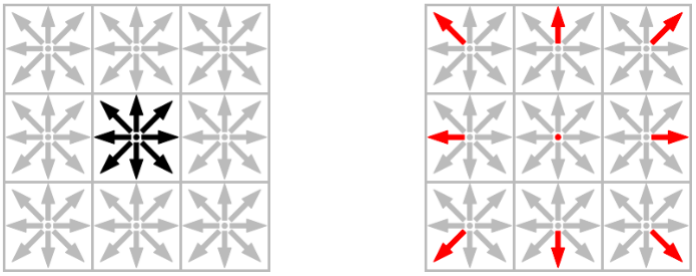


Figure 2.3: (Left) Pre-collision distributions and (Right) post-collision distribution in "Push" streaming [13]

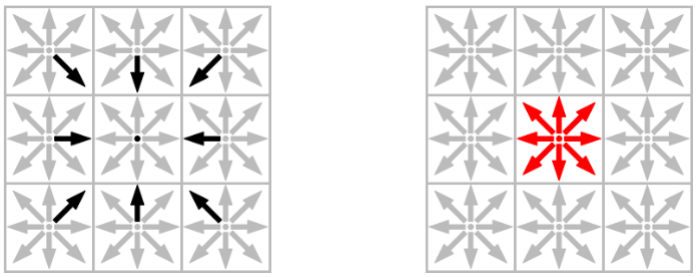


Figure 2.4: (Left) Pre-collision distributions and (Right) post-collision distribution in "Pull" streaming [13]

As depicted in figures 2.3 and 2.4, the left part reads pre-collision distribution values to an array, and then the post-collision values are written to another array in the respective streaming method. These "push" and "pull" methods are repeated for each lattice site at each time step, making LBM a computationally intensive process. The choice of the equilibrium distribution function and collision model depends on the specific problem being simulated and the desired properties of the fluid behaviour.

2.1.3 Macroscopic Relations

Compressible fluid flow simulations need to solve the Navier-Stokes-Fourier equations, which include mass, momentum and energy equations. The moments of the Boltzmann equation can recover the macroscopic properties of these three conservation equations. In the continuous Boltzmann equation, the moments are defined as the weighted integrals of f in velocity space. To recover the conservation laws on the macroscopic level, the first three terms of the Hermite series expansion are sufficient to recover the macroscopic laws for hydrodynamics. This allows for a significant reduction of numerical effort [1].
Since the discretised Boltzmann equation is used in the consistent LBM method, the moments related to mass, momentum and energy can be stated as follows.

$$\text{Mass :} \qquad \sum_{i=0}^{Q-1} f_i = \rho \tag{2.2}$$

$$\text{Momentum :} \qquad \sum_{i=0}^{Q-1} c_\alpha \cdot f_i = \rho \cdot u_\alpha \tag{2.3}$$

$$\text{Energy :} \qquad \sum_{i=0}^{Q-1} c_\alpha \cdot c_\alpha \cdot f_i = 2 \cdot \rho \cdot T + u_\alpha \cdot u_\alpha \text{ [14]} \tag{2.4}$$

Equations 2.2 and 2.3 represent the mass and momentum conservation, while equation 2.4 is for the energy equation. In above equations, ρ, T and u represent the non-dimensional density, non-dimensional temperature and non-dimensional velocities respectively. The equation of state used in LBM is stated in equation 2.5, where p denotes the non-dimensional pressure. Furthermore, the speed of sound can also be related to the lattice reference temperature, T_0 as in equation 2.6.

$$p = \rho \cdot {c_s}^2 \tag{2.5}$$

$$T_0 = {c_s}^2 \tag{2.6}$$

2.2 Collision Operators; Towards Compressible LBM

The collision operator is responsible for updating the f at each lattice site during the collision step of the LBM algorithm. This step models the particle interactions and the relaxation of the distribution functions toward local equilibrium. The choice of collision operator in the LBM can have a significant impact on the accuracy, stability, and efficiency of the simulation. In this section, a concise description of collision operators that can be used to simulate incompressible and compressible fluid flows is summarised as follows.

2.2.1 Single Relaxation Time (SRT) Collision Operator

Among the many collision operators, the SRT collision operator, also known as the Bhatnagar-Gross-Krook (BGK) operator [15], is widely used in a vast range of fluid simulations considering its simplicity and efficiency within the LBM community. The BGK collision operator models the tendency of f_i to approach its equilibrium state f^{eq} after time τ, where τ is known as the relaxation time [1]. The ${f_i}^{eq}$ used in standard LBM which is in the $\mathcal{O}(u^2)$ can be stated as equation 2.8.

$$\Omega_i = -\frac{f_i - {f_i}^{eq}}{\tau} \tag{2.7}$$

$${f_i}^{eq}(x,t) = w_i \cdot \rho\left(1 + \frac{\vec{u} \cdot \vec{c_i}}{{c_s}^2} + \frac{(\vec{u} \cdot \vec{c_i})^2}{2 \cdot {c_s}^4} + \frac{\vec{u} \cdot \vec{u}}{2 \cdot {c_s}^2}\right) \tag{2.8}$$

Chapman-Enskog analysis is used to bridge the gap between LBM and NSE. According to that, the macroscopic relation to kinematic viscosity ν can be defined as equation 2.9. With the above definitions, LBM with the BGK collision operator is also known as the Lattice BGK (LBGK).

$$\nu = {c_s}^2\left(\tau - \frac{\Delta t}{2}\right) \tag{2.9}$$

However, for large Reynold numbers as well as for high Mach numbers and high-temperature variations the BGK operator is vulnerable to severe numerical instabilities [16], [1]. Further, it is limited to simulations of flows with Prandtl number, Pr = 1. In compressible LBM literature, there are some implementations using BGK collision operator [17], [6]. However, these implementations are based on lattice stencils with extended neighbourhoods. One of the problems with lattice with extended neighbourhoods is the boundary treatment when there are complex simulation domains.

2.2.2 Multiple Relaxation Time (MRT) Collision Operator

The SRT-BGK operator uses only one relaxation time constant for all the moments in a given lattice configuration. Therefore, to mitigate the drawbacks of SRT-BGK, the MRT operator uses multiple relaxation time constants for each moment. The MRT collision operator's fundamental concept is a transformation matrix M-based mapping from population to moment space. This enables moments to be relaxed with individual rates as opposed to populations. After relaxation, the moments are transformed back to the population space, where standard streaming operations are executed as usual [1]. The general form of the collide and stream algorithm with the MRT collision operator can be expressed as follows.

$$f_i(x + c_i\delta t, t + \delta t) = f_i(x,t) - \boldsymbol{M}^{-1}\boldsymbol{S}\boldsymbol{M}[f_i(x,t) - {f_i}^{eq}(x,t)] \tag{2.10}$$

In equation 2.10, the **M** and **S** indicate the transformation matrix and relaxation matrix respectively. **S** is the diagonal matrix which contains all the relaxation time constants. The transformation matrix has the property of $\mathbf{M}\mathbf{M}^{-1} = \mathbf{I}$, where **I** being the identity matrix. SRT-BGK collision operator can be also seen as a special case of MRT collision operator where the matrix **S** contains only one relaxation time. Further, TRT, Two Relaxation Time collision operator is also derived from MRT collision operator, where in TRT operator, there are two relaxation times. One relaxation time controls the kinematic viscosity while the other is the tunning relaxation time [1].
Several collision operators have emerged, depending on the choice of the transformation matrix, **M**. For instance, the MRT method is applied to Hermite moment space, central-moments space (referred to as the cascaded model), and cumulant space. These variants are designed for specific applications and flow scenarios [16]. The MRT collision can overcome the stability and accuracy issues in the BGK operator. Further, one of the main advantages of the MRT operator is that it provides the capability to adjust shear and bulk viscosities separately. Enhancing the bulk viscosity can frequently enhance stability by reducing unresolved hydrodynamic effects [1].
However, one of the major drawbacks of the MRT operator is the reduced computational efficiency. Additionally, the MRT operator can be tedious to adapt to compressible flow phenomena considering standard lattice configurations, being computationally inefficient.

2.2.3 Entropic Collision Operator

For the previously mentioned collision operators, the discrete equilibrium distribution ${f_i}^{eq}$ is derived using a polynomial approach. In contrast, for the entropic collision operators, the ${f_i}^{eq}$ is approximated by minimizing a convex discrete entropy functional under a certain number of constraints [16]. The entropy function consistently used in the literature has the form of equation 2.11.

$$H(f) = \sum_{i=0}^{Q-1} f_i \cdot ln\left(\frac{f_i}{\kappa}\right) \tag{2.11}$$

To obtain the ${f_i}^{eq}$, equation 2.11 is minimized according to the condition:

$$\frac{\partial H}{\partial f_i} = 0 \tag{2.12}$$

Here, H is the discrete entropy function, which is also defined as the H-function. According to Boghosian [18], the Lyapunov functional is analogous to Boltzmann's H-function, resulting in unconditional numerical stability. In the literature, several values for κ have been suggested,

and this will be further discussed in the next chapter. The polynomial equilibria have two key limitations: limited positivity, where certain terms become negative at high Mach numbers, and inaccuracies at high flow velocities. The loss of positivity not only compromises the physical interpretation of populations but also contributes to numerical instabilities in the scheme [5]. Therefore, in this thesis scope, the entropic collision model is adopted to simulate compressible fluid flows, where the Mach number is greater than one. This will be discussed again in the next chapter when formulating the minimization problem. The LBM with the entropic collision is commonly known as the Entropic LBM (ELBM). In compressible flow simulations and in thermal simulations ELBM has been proven to be more stable and accurate while being comparatively computationally robust.

2.3 Limitations of Standard LBM in Compressible Fluid Domain

This section aims to provide a concise assessment of the LBM in the specific context of simulating compressible flows with higher Mach numbers. While LBM offers various advantages, it is essential to acknowledge its limitations to guide researchers and engineers in choosing appropriate computational tools for their applications.

2.3.1 Weakly Compressibility

According to Chapman-Enskog analysis, it can be shown that standard lattices used in space discretisation are not sufficiently isotropic to accurately describe the third-order velocity moment. This leads to an $\mathcal{O}(u^3)$ error term in the viscous stress tensor. LBM does not precisely solve the NSE. While this discrepancy is insignificant at low flow velocities, it becomes crucial at high Mach numbers, introducing errors and undermining Galilean invariance in the macroscopic equations [1].
Especially, in the case of compressible flow problems involving shock formations, and flows in transonic or supersonic regimes, the standard LBM with space discretisation using standard stencils cannot resolve the Navier-Stokes Fourier equations. The equation 2.8 can be used to resolve the NSE. However, due to $\mathcal{O}(u^3)$ error, it is limited to small Mach numbers. Therefore, the standard LBM is typically known as a "weakly compressible" solver for NSE. This can be illustrated by using a simple example case for a shock tube with a small pressure difference.

2.3.2 Case Study: Investigation of Stability Region of Standard LBM

Although the proper shock tube simulations will be covered in the next chapters, in this section, a shock tube with a small pressure difference is investigated just to observe the stability margin and the effect of weakly compressibility of the standard LBM. Since the pressure difference is small, let us assume that the viscosity remains constant throughout the simulation time. Figure 2.5 represents the schematic diagram of a shock tube. The left side has a slightly higher pressure compared to the right side of the tube.

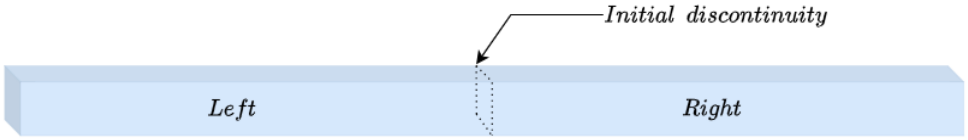


Figure 2.5: Schematic diagram of a shock tube.

	Left	**Right**
density (ρ)	$\rho = p/{c_s}^2$	$\rho = p/{c_s}^2$
velocity (u)	0	0
Temperature (T)	constant	constant

Table 2.2: Macroscopic properties inside the shock tube

The pressure inside the shock tube is defined according to the equation of state 2.5. The density inside the tube is varied in between 0.25 – 1.0, considering different relaxation time, τ. Then the maximum achievable lattice velocity inside the shock tube is investigated. As mentioned before, the τ can be assumed as a constant due to the consideration of small pressure differences. For this study, the D1Q3 lattice stencil with "push" streaming is used for the standard LBM algorithm implementation.
Starting from the sufficient stability condition $\tau/\Delta t > 1/2$ the value of τ is gradually increased to obtain a stable velocity profile for a particular pressure difference. Note that here the $\Delta t = 1$. The stability region for the present case is shown in figure 2.6.

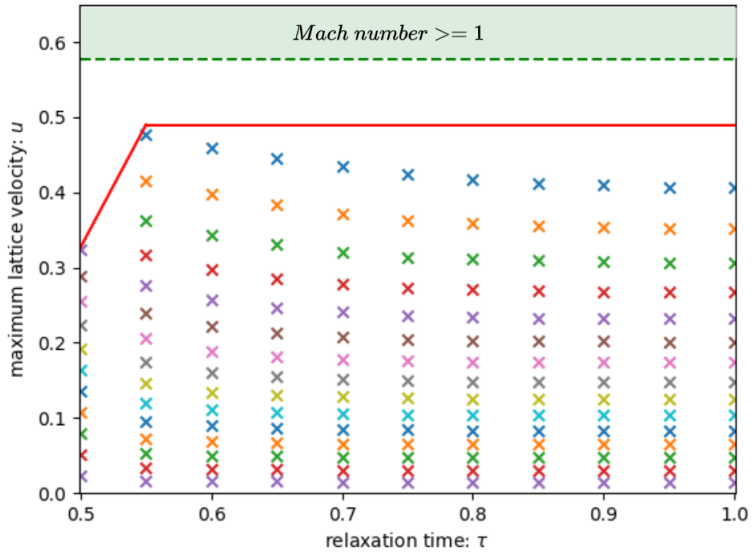


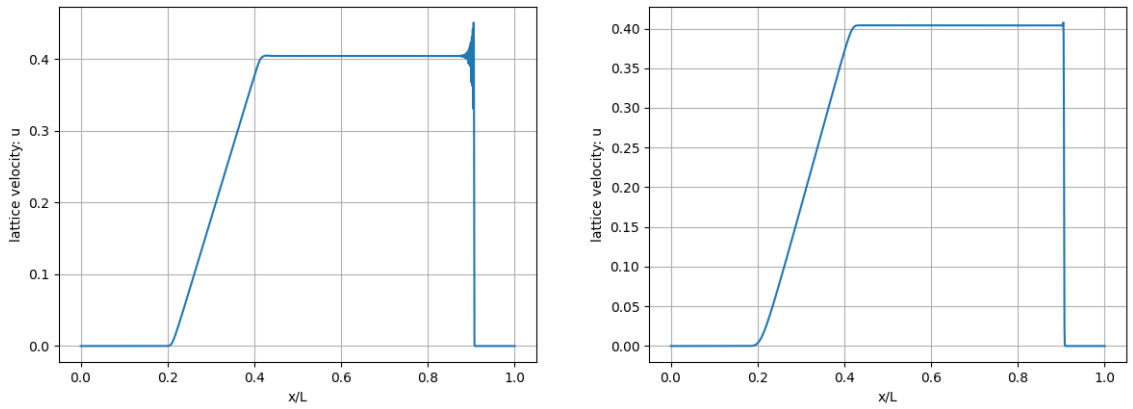
Figure 2.6: Stability margin for standard LBM for a shock tube simulation with small pressure difference

The cross points in the figure 2.6 represent the achieved maximum velocity for a given pressure difference. All the maximum velocities are below the upper limit, which is the Mach number of 1.0. The Mach number in standard LBM is defined by equation 2.13.

$$Ma = u/c_s \tag{2.13}$$

According to the observation, the maximum velocity has an exponential decaying pattern with the increase of τ when it reaches the upper limit, which is shaded in green colour in the above figure 2.6. One might argue that there is one achievable maximum velocity for $\tau = 0.55$ by considering the top most 'x' marks in the above figure 2.6. However, this is due to the spurious oscillation produced by the discontinuities inside the shock tube, as shown in figure 2.7a. This

can be reduced by increasing the τ, as shown in figure 2.7b.
Therefore, the stability region for standard LBM can be identified as the area below the red line in figure 2.6. Although the simulation is stable, it is always necessary to consider the correct representation of the physics in the simulation. Even though it is obvious that the standard weakly compressible LBM cannot capture the shock tube physics, a comparison of the velocity profile of an analytical shock tube with equivalent properties is depicted in figure 2.8.



(a) $\tau = 0.55$ (b) $\tau = 0.9$

Figure 2.7: Velocity profile inside the shock tube against normalized length

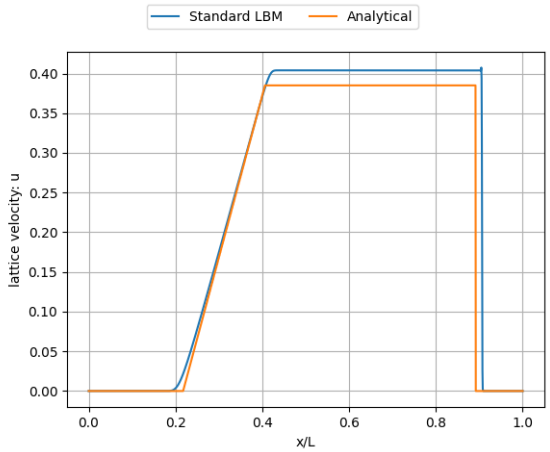


Figure 2.8: Comparison of analytical and LBM velocity profiles inside the shock tube

Therefore, to increase the stability and the accuracy of the fluid simulation with higher Mach numbers, it is vital to adopt an LBM method with a proper collision model. Furthermore, it is important to retain the native LBM collide and stream algorithm while simulating compressible fluid flows to preserve its parallelizing capability. Therefore, in the upcoming chapter, a compressible LBM method will be introduced and discussed in detail.

Chapter 3

Compressible Lattice Boltzmann Method

In this chapter, the background of the compressible LBM implemented in this research is widely discussed. As mentioned in the previous chapter, the Entropic Lattice Boltzmann Method (ELBM) is adopted for simulating compressible fluid flow problems. This chapter is structured as follows: it begins with a concise discussion of the theoretical background of the two-population methods. Next, the minimization problem present within the ELBM is discussed. At the end, the implementation of the present compressible LBM is discussed, along with a discussion of initial and boundary conditions in implementing the compressible LBM.

3.1 Double Distribution Function (DDF) LBM

In the first chapter, it has been mentioned that there are some LBM implementations considering the compressible fluid flows, coupled with FDM or FVM [2],[3],[4],[19]. However, it is worth mentioning that these implementations did not match the standard LBM collide and stream framework. Therefore, adopting the main collide and stream approach to simulate compressible flows is one of the main focuses within the LBM community. Consequently, based on that, several implementations have been developed during the past few years [5],[8],[6],[10]. In the following sections, the key points of these recent implementations will be discussed to align with the present scope of this thesis.

3.1.1 Double Distribution Function (DDF) Approach

The Double Distribution Function (DDF) approach, also known as the two-population method, is a popular choice for thermal lattice Boltzmann simulations, and later, for compressible lattice Boltzmann simulations [20]. In this approach, two sets of discrete populations, denoted as f and g, are utilized at a lattice node. However, as mentioned in the second chapter, standard lattice stencils, along with the SRT-BGK collision operator, have some limitations when simulating compressible fluid flow problems. Therefore, a multispeed lattice stencil should be adopted when using the DDF approach with a BGK collision operator. Furthermore, the use of DDF with a BGK collision operator restricts the flow problems to a Prandtl number, Pr=1 [6]. To overcome this limitation, several advanced collision models were initially adopted for monoatomic cases and later extended to polyatomic gases [17]. In the literature, one of the first compressible LBM implementations based on the collide and stream algorithm

with the DDF approach for $DdQ7^d$ stencils can be described as follows [21].

$$f_i(x + c_i\delta t, t + \delta t) = f_i(x,t) + \alpha\beta_1(f_i^* - f_i) + 2\beta_2({f_i}^{eq} - f_i^*) \tag{3.1}$$
$$g_i(x + c_i\delta t, t + \delta t) = g_i(x,t) + \alpha\beta_1(g_i^* - g_i) + 2\beta_2({g_i}^{eq} - g_i^*) \tag{3.2}$$

In above equations 3.1 and 3.2, the f, g represent the discrete distribution functions while superscripts '$*$' and 'eq' relates to the quasi-equilibrium discrete distribution and equilibrium distributions. β's represent the time dependent relaxation times and the α is maximal over relaxation parameter which can be obtained by finding the positive root of the equation 3.3.

$$H(f + \alpha(f^{eq} - f)) = H(f) \tag{3.3}$$

However, while the above-mentioned compressible LBM can facilitate a large number of compressible fluid flow scenarios and is considered the most accurate compressible LBM based on the collide and stream algorithm, the existence of positive roots for equation 3.3 is a necessity. Therefore, it has been noted as one of the limitations of this collision model in later literature [10],[6]. Additionally, practical limitations of the $DdQ7^d$ stencil, particularly in three-dimensional flow problems, have led Latt et al. [6] and Coreixas et al. [17] to demonstrate that accurate predictions of the Navier-Stokes-Fourier equations can be achieved using $D3Q39$ for three-dimensional cases and $D2Q21$ for two-dimensional problems. It is important to note that all of the above-mentioned LBMs use extended lattices. Saadat et al. [8] proposed a DDF-based compressible LBM method for standard lattice stencils. In this approach, with the aid of suitable correction terms, the model accurately reconstructs the complete Navier-Stokes-Fourier equations under hydrodynamic conditions. The two-population model used in this approach is described in equations 3.4 and 3.5.

$$f_i(x + c_i\delta t, t + \delta t) = f_i(x,t) + \omega({f_i}^{eq} - f_i) + \phi_i \tag{3.4}$$
$$g_i(x + c_i\delta t, t + \delta t) = g_i(x,t) + \omega(g_i^{eq} - g_i) + (\omega_1 - \omega)({g_i}^* - g_i) \tag{3.5}$$

Here, the f, g has the same definition as the above equations 3.1 and 3.2, where f population relates to the mass and momentum while g is responsible for solving total energy consevation. The ω and ω_1 is time dependent relaxation time relates to dynamic viscosity and thermal conductivity. However, adding non local correction terms to the 3.4 and also for the g_i^* has made slight deviation from collide and stream algorithm. Further, the corection terms need to be approximated through FDM. While, the suggested model proven accurate and stable predictions for compressible flows, it may have some complication if the simulation domain is complex and three dimensional. Based on above model for standard stencils, Tran et al. [10] have suggested another DDF approach. Here, the collision model has the same form as in [8] without the correction terms. In this thesis, the model proposed by [10] is investigated and implemented. The collision model in [10] can be stated as follows.

$$f_i(x + c_i\delta t, t + \delta t) = f_i(x,t) + \omega({f_i}^{eq} - f_i) \tag{3.6}$$
$$g_i(x + c_i\delta t, t + \delta t) = g_i(x,t) + \omega(g_i^{eq} - g_i) + (\omega - \omega_1)({g_i}^* - g^{eq}) \tag{3.7}$$

All the notations in equations 3.6 and 3.7 have the same definitions as those in equations 3.4 and 3.5. The only noticeable difference between equation 3.5 and equation 3.7 is the change of g_i to g^{eq} and the change of ω_1 into ω in equation 3.5. However, according to the kinetic theory of gases, it can be shown that $\sum_{i=0}^{Q-1} g_i = \sum_{i=0}^{Q-1} g_i^{eq}$. Furthermore, based on observations conducted within the scope of this thesis, it has been found that these changes do not lead to deviations in the obtained results. The main difference between the above collision model and previous models lies in the approximation of g^{eq} related to total energy conservation, which

will be discussed in the next section. The relationship between macroscopic properties, as considered in the present model, can be described as follows.

$$\text{Mass} : \sum_{i=0}^{Q-1} f_i = \sum_{i=0}^{Q-1} f_i^{eq} = \rho \tag{3.8}$$

$$\text{Momentum} : \sum_{i=0}^{Q-1} c_\alpha \cdot f_i = \sum_{i=0}^{Q-1} c_\alpha \cdot f_i^{eq} = \rho \cdot u_\alpha \tag{3.9}$$

$$\text{Energy} : \sum_{i=0}^{Q-1} g_i = \sum_{i=0}^{Q-1} g_i^{eq} = 2 \cdot \rho \cdot E \tag{3.10}$$

$$\text{Temperature} : T = \frac{1}{c_v}\left(E - \frac{u^2}{2}\right) \tag{3.11}$$

The term u^2 denotes the velocity magnitude. According to the Chapman-Enskog analysis, a connection between compressible macroscopic hydrodynamic properties and the relaxation times can be derived. Additionally, as the collision model also supports variable Prandtl numbers (Pr), the definition of Pr can be utilized to establish a connection between thermal conductivity.

$$\text{Dynamic viscosity} : \mu = \left(\frac{1}{\omega} - 0.5\right)\rho T \tag{3.12}$$

$$\text{Thermal conductivity} : k = c_p\left(\frac{1}{\omega_1} - 0.5\right)\rho T \tag{3.13}$$

$$\text{Pr} : Pr = \frac{c_p \mu}{k} \tag{3.14}$$

In equation 3.13 and 3.14, the c_p represents the specific heat at constatn pressure. The c_p and the c_v, specific heat at constant volume can be connected by heat capacity ratio, γ which is also known as the adiabatic constant as follows.

$$c_p = c_v + 1 \tag{3.15}$$

$$\gamma = c_p / c_v \tag{3.16}$$

3.2 Discrete Equilibrium Distribution g^{eq}

Most of the time, the equilibrium distribution is represented as a polynomial function in LBM. As mentioned earlier, one of the main drawbacks of this approach is the lack of positivity for higher Mach numbers, which can lead to instabilities. However, the entropic way of constructing the equilibrium distribution offers better positivity for high Mach number compressible flows compared to the polynomial approach. The starting point for this is equation 2.11. Equation 2.11 is then minimized according to a carefully chosen set of constraints defined by the moments of the Maxwell-Boltzmann distribution. In previous literature, in the context of obtaining a discrete equilibrium distribution, most methods minimized equation 2.11 with respect to the distribution related to mass and momentum, denoted as f. However, in [10], the discrete H-function is minimized with respect to the g distribution. Interestingly, the minimization is also subject to a few constraints, namely the total energy constraint in equation 3.18 and the trace of the third-order Maxwell-Boltzmann moment in equation 3.19. The present minimization problem can be described as follows.

$$\text{Minimize} : H = \sum_{i=0}^{Q-1} g_i \cdot ln\left(\frac{g_i}{\kappa}\right) \tag{3.17}$$

subjected to:

$$\text{Energy} : \sum_{i=0}^{Q-1} g_i = 2\rho E \tag{3.18}$$

$$\text{Trace of third order Maxwell-Boltzmann moment} : q_\alpha^{eq} = \sum_{i=0}^{Q-1} c_{i\alpha} g_i^{eq} = 2\rho u_\alpha (E + T) \tag{3.19}$$

The method of minimization employs the Lagrangian multiplier technique. A detailed derivation for the present case can be found in the Appendix A.1. According to the minimization, g_i^{eq} takes the following form, employing Einstein index notation.

$$g_i^{eq} = \kappa \; exp(-1(1 + \lambda + \mu_\alpha c_{i\alpha})) \tag{3.20}$$

In equation 3.20, λ and μ represent the Lagrangian multipliers. However, parameter κ has a different selections in literature. Initially the κ is replaced by ρw_i in multiple publications [21],[7],[5],[8]. However, if the $\kappa = \rho w_i$, the form of the equation 3.19 has adopted differently as follows.

$$g_i^{eq} = \rho w_i(T) \; exp(\lambda + \mu_\alpha c_{i\alpha}) \tag{3.21}$$

In contrast, in [6], it has been mentioned that a possible choice for κ is ρ for fluid flow applications and $\kappa = 1$ for rarefied gas simulations. In these cases, the form of equation 3.20 remains unchanged for those selections and simulations were carried out using lattices with extended neighborhoods. To be consistent with standard lattices, the form in equation 3.21 is used in this thesis scope. Another important point to note is the selection of weights in equation 3.21. In standard LBM, fixed constant values are used for the respective lattice links according to the lattice configuration. However, in this case, the weights are temperature-dependent. In each time step, the value of a particular weight changes. The derivation of temperature dependence can be found in [22]. If the temperature of the flow is T, the respective weight of a lattice link can be expressed as the following algebraic product. In equation 3.23, w_{ix}, w_{iy}, w_{iz} correspond to the respective discrete lattice velocity.

$$w_i(T) = w_{ix} \cdot w_{iy} \cdot w_{iz} \tag{3.22}$$

$$\tag{3.23}$$

where,

$$w_{(0)} = 1 - T \tag{3.24}$$

$$w_{(+1)} = T/2 \tag{3.25}$$

$$w_{(-1)} = T/2 \tag{3.26}$$

In above formulas, the $0, +1, -1$ represent the respective coordinate indicies of interested lattice stencil. The tempertaure dependent weights for the $D2Q9$ (Refer the figure: 2.2b) standard lattice is shown in table 3.1.

Lattice link	$D2Q9$ **lattice indicies**	**Temperature dependent weights** $w_i(T)$
0	$(0,0)$	$(1-T)(1-T)$
1	$(1,0)$	$(T/2)(1-T)$
2	$(0,1)$	$(1-T)(T/2)$
3	$(-1,0)$	$(T/2)(1-T)$
4	$(0,-1)$	$(1-T)(T/2)$
5	$(1,1)$	$(T/2)(T/2)$
6	$(-1,1)$	$(T/2)(T/2)$
7	$(-1,-1)$	$(T/2)(T/2)$
8	$(1,-1)$	$(T/2)(T/2)$

Table 3.1: Temperatre dependent weights for D2Q9 stencil

Finally, the g_i^{eq} can be estimated by finding the Lagrangian multipliers in equation 3.21. To find the Lagrangian multipliers, this thesis employs the Newton-Raphson scheme. In each Newton-Raphson iteration, the previous values of the Lagrangian multipliers are used for the next time step. According to the chosen constraints, the Newton-Raphson algorithm needs to find the inverse of the Jacobian matrix in each time step. Generally it is denoted as $\boldsymbol{J}$ and has dimensions $m \times m$, where m is the size of the matrix. For instance, in a one-dimensional flow problem, the constraints in equations 3.18 and 3.19 provide two equations. Similarly, in a two-dimensional flow problem, the same constraints provide three equations. Therefore, depending on that, for a one-dimensional flow problem, the size of the $\boldsymbol{J}$ is $m = 2$, and for a two-dimensional problem, the size of the $\boldsymbol{J}$ is $m = 3$. In general, it can be seen that, according to the dimensionality D, and according to the current constraints in equations 3.18 and 3.19, the Jacobian matrix has the form of $\boldsymbol{J_{(D+1)\times(D+1)}}$. However, in ELBM methods that use lattices with extended neighborhoods, more constraints are required, increasing the size of m. Therefore, in multispeed lattices, finding the inverse requires more computaional effort compared to standard stencils. One of the main issues with ELBM methods is the potential for getting a singularity in the matrix when finding the inverse. However, root-finding methods that rely on exact representations of the Jacobian matrix result in expanded stability intervals. Moreover, as the lattice size increases, the stability region of the LBM also expands [17].

3.3 Definition of Discrete Equilibrium Distribution f^{eq} and Quasi-Equilibrium g^*

Compared to the polynomial approximation of f^{eq} in standard LBM, f^{eq} is again approximated using an entropic estimation. In this case, equation 2.11 is minimized with respect to f_i subject to constraints 2.2 – 2.3 and the equilibrium pressure tensor of the consistent lattice Boltzmann method, as given in equation 3.27.

$$\text{Equilibrium pressure tensor} : P_{\alpha\beta}^{eq} = \sum_{i=0}^{Q-1} c_{i\alpha} c_{i\beta} f_i^{eq} = \rho T \delta_{\alpha\beta} + \rho u_\alpha u_\beta \text{ [14]} \tag{3.27}$$

In the above equation 3.27, the symbols α and β are related to the Einstein index notation, and $\delta_{\alpha\beta}$ represents the Kronecker delta. As in the previous section, after utilizing Lagrangian

multipliers, the formulation for f^{eq} can be obtained as follows, with $\kappa = w_i$

$$f_i^{eq} = w_i \; exp(\lambda + \mu_\alpha c_{i\alpha} + \zeta_{\alpha\beta} c_{i\alpha} c_{i\beta} + \gamma c_i^2) \tag{3.28}$$

As mentioned in [14], equilibria with non-zero velocities are obtained by perturbing the zero-velocity state, and the equilibrium distribution functions f_i^{eq} are expressed as a series based on momentum, expanded up to $\mathcal{O}(u^4)$. Then the f_i^{eq} can also be represented as follows.

$$f_i^{eq} = \rho \; \phi_{ix} \cdot \phi_{iy} \cdot \phi_{iz} \tag{3.29}$$

where,

$$\phi_{(0)} = 1 - (u_\alpha^2 + T) \tag{3.30}$$

$$\phi_{(+1)} = \frac{u_\alpha + u_\alpha^2 + T}{2} \tag{3.31}$$

$$\phi_{(-1)} = \frac{-u_\alpha + u_\alpha^2 + T}{2} \tag{3.32}$$

Therefore, in this case, there is no need to use another Newton-Raphson scheme to estimate the f_i^{eq}. Here, the same convention as in temperature-dependent weights applies when determining the f_i^{eq} of a particular lattice link of a standard stencil.
The quasi-equilibrium g^* incorporates the pressure tensor into the present collision model. According to the literature, a second-order approximation for the entropic equilibrium can be derived from Grad's distribution. This approximation can also be used to construct the equilibrium and the quasi-equilibrium. The quasi-equilibrium approximation, alternatively referred to as the entropic Grad method, maximum entropy method, or generalized canonical ensemble, represents a significant extension of Grad's method. The general form of quasi-equilibrium can be stated as follows [7].

$$F_i = W_i \left(M_0 + \frac{M_\alpha v_{i\alpha}}{T_0} + \frac{(M_{\alpha\beta} - M_0 T_0 \delta_{\alpha\beta})}{2T_0^2} \left(v_{i\alpha} v_{i\beta} - T_0 \delta_{\alpha\beta} \right) \right) \tag{3.33}$$

$$\tag{3.34}$$

In the context of the provided equation, the terms M_0, M_α and $M_{\alpha\beta}$ correspond to the zeroth-order, first-order, and second-order moments. When considering the discrete population g, the respective relations can be stated as follows,

$$M_0 = 2\rho E \tag{3.35}$$

$$M_\alpha = q_\alpha - 2u_\beta \left(P_{\alpha\beta} - P_{\alpha\beta}^{eq} \right) \tag{3.36}$$

$$M_{\alpha\beta} = R^{eq} \tag{3.37}$$

Here, the terms q_α and R^{eq} represent the energy flux tensor and contracted fourth-order moments, respectively. Considering the above definitions, the equilibrium g^{eq} and the quasi-equilibrium for g^* can be expressed as follows,

$$g_{eq} = W_i \left(2\rho E + \frac{q_\alpha^{eq} c_{i\alpha}}{T} + \frac{(R^{eq} - 2\rho E T \delta_{\alpha\beta})}{2T^2} \left(c_{i\alpha} c_{i\beta} - T\delta_{\alpha\beta} \right) \right) \tag{3.38}$$

$$g_i^* = W_i \left(2\rho E + \frac{(q_\alpha + 2u_\beta (P_{\alpha\beta} - P_{\alpha\beta}^{eq})) c_{i\alpha}}{T} + \frac{(R^{eq} - 2\rho E T \delta_{\alpha\beta})}{2T^2} \left(c_{i\alpha} c_{i\beta} - T\delta_{\alpha\beta} \right) \right) \tag{3.39}$$

Therefore, the g_i^* can be approximated according to equation 3.40, and the required pressure tensor $P_{\alpha\beta}$ can be approximated according to equation 3.41.

$$g_i^* = g_i^{eq} + W_i \left(\frac{2u_\beta (P_{\alpha\beta} - P_{\alpha\beta}^{eq}) c_{i\alpha}}{T} \right) \tag{3.40}$$

$$P_{\alpha\beta} = \sum_{i=0}^{Q-1} c_{i\alpha} c_{i\beta} f_i \tag{3.41}$$

Considering all the relevent approximations, In next section the implementation of the compressible ELBM will be discussed.

3.4 Implementation of Compressible ELBM

In this section, we delve into the code implementation, a fundamental component of this thesis. Main objective of this section is to provide concise description of code implementation for the developed compressible LBM solver. To begin with, lets summarize all the neccessary equations required for the implementation.

Collision model for f (3.6) : $f_i(x + c_i\delta t, t + \delta t) = f_i(x,t) + \omega({f_i}^{eq} - f_i)$

Collision model for g (3.7) : $g_i(x + c_i\delta t, t + \delta t) = g_i(x,t) + \omega(g_i^{eq} - g_i) + (\omega - \omega_1)({g_i}^* - g^{eq})$

Equilibrium distribution f^{eq} (3.29) : $f_i^{eq} = \rho\ \phi_{ix} \cdot \phi_{iy} \cdot \phi_{iz}$

Equilibrium distribution g^{eq} (3.21) : $g_i^{eq} = \rho w_i(T)\ exp(\lambda + \mu_\alpha c_{i\alpha})$

Quasi-equilibrium distribution g^* (3.40) : $g_i^* = g_i^{eq} + W_i \left(\frac{2u_\beta (P_{\alpha\beta} - P_{\alpha\beta}^{eq}) c_{i\alpha}}{T} \right)$

Pressure tensor P (3.41) : $P_{\alpha\beta} = \sum_{i=0}^{Q-1} c_{i\alpha} c_{i\beta} f_i$

Relaxation factor ω (3.12) : $\omega = \frac{1}{\left(\frac{\mu}{\rho T + 0.5} \right)}$

Relaxation factor ω_1 (3.13) : $\omega_1 = \frac{1}{\left(\frac{k}{c_p \rho T + 0.5} \right)}$

Prandtl number Pr (3.14) : $Pr = \frac{c_p \mu}{k}$

For the implementation of the current compressible LBM solver, C++17 is used as the programming language. In addition to that, Python is used to visualize the obtained data, apart from Paraview. The capabilities of C++ Object-Oriented Programming (OOP) are utilized to represent the lattice nodes and grid implementation. The implementation consists of two main classes, each containing all the attributes and methods according to the rule of three, as shown in figure 3.1. All the arrays are one-dimensional, and operator overloading is used to access the grid points accordingly, depending on the dimensionality of the computational grid.

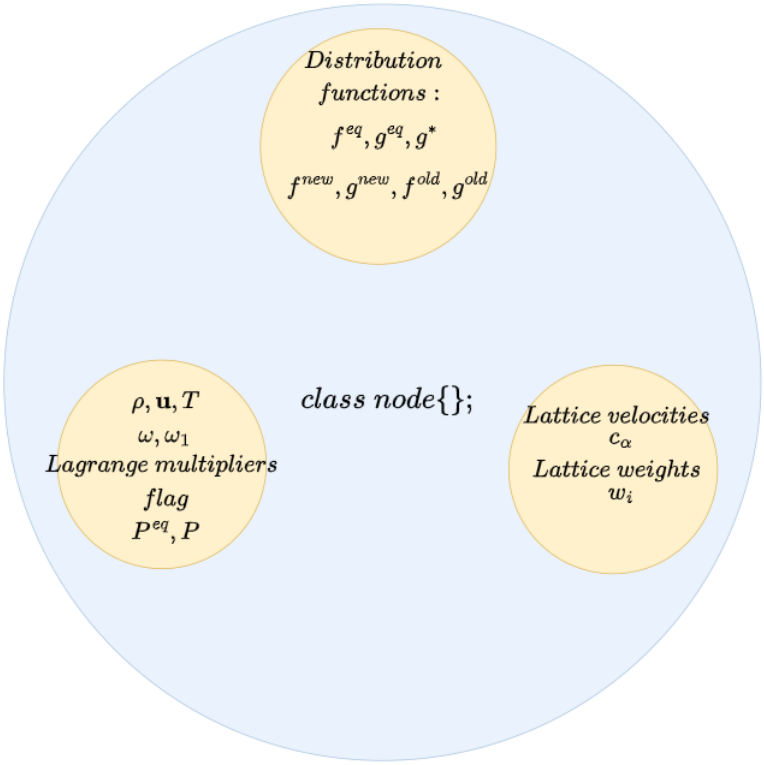


Figure 3.1: C++ node class attributes to represent a lattice node

In the present implementation, the "pull" streaming is incorporated. Therefore, as shown in figure 3.1, the superscripts "old" and "new" in discrete distributions relate to the pre-collision and post-collision arrays. The flag variable is an integer that helps define boundary conditions appropriately. Further, the Lagrangian multiplier also represents an array according to the dimensionality of the problem. The pressure tensor P is also an array where the number of elements depends on the dimensionality of the problem. For instance, in two-dimensional compressible flow simulation, each lattice node has a two-by-two (2×2) pressure tensor, and for a three-dimensional case, it is a three-by-three (3×3) pressure tensor. The discrete velocities c and lattice weights w are arrays of doubles according to the chosen lattice configuration. The other main class is used to represent the computational grid for the simulation, named as the "grid" class (see Appendix: B.1). Here, the main attributes are the element size along each spatial axis and a pointer to a node object.
Compared to standard LBM, all the relaxation time constants are now time-dependent, and therefore, they should be updated during each time step. Further, the lattice weights w_i are also temperature-dependent, and therefore, they should also change with time steps. The computational grid required for complex simulation domains is generated separately, and then the generated grid is read into a grid object before all the relevant boundary information is specified according to the flag variable. All the post-processing data is written into separate folders as .vtk files or .out files containing the post-processing data at a particular time step. The required input parameters for a particular simulation are fed into the solver using a .in file. Therefore, with all the information up to now, the preliminary algorithm for the compressible LBM can be stated as shown in Algorithm 1. The remaining topics in this section will discuss the implementation details of the important kernels in Algorithm 1.

Algorithm 1: Compressible LBM algorithm for subsonic fluid flows.

Data: Read computational grid and fluid properties;
Initialization: $\rho, \mathbf{u}, T$ and set temperature dependent weights;
Calculate: f^{eq};
Estimate Lagrangian multipliers $\leftarrow$ Newton-Raphson ;
Calculate: g^{eq} and Initialize g, f;
for $(int\ i = 0; i < Number\ of\ time\ steps; ++i)$
{
    Calculate $\rho, \mathbf{u}, T$ and set temperature dependent weights;
    Calculate time dependent relaxation times;
    Write .vtk files for post-processing;
    Calculate f^{eq};
    Estimate Lagrangian multipliers $\leftarrow$ Newton-Raphson;
    Calculate g^{eq};
    Calculate pressure tensor;
    Calculate quasi-equilibrium distribution g^{*};
    Collide;
    Set boundary conditions;
    Stream;
    Swap f^{new} and f^{old};
}

3.5 Initialization and Boundary Conditions

3.5.1 Initialization

Initialization is a critical phase in LBM that lays the foundation for the entire computational process. It involves setting up fluid parameters and conditions that define the fluid flow problem to be solved, ensuring accuracy and reliability in the subsequent steps of the simulation. In this section, the key aspects of initialization used in the current compressible LBM solver are discussed, from defining the computational domain to initial fluid properties. LBM is a time-dependent scheme. Therefore, unsteady flow simulations are mostly preferred compared to steady flow simulations using LBM [1]. Since compressible fluid simulations fall into the category of unsteady flow scenarios, they require a large number of time steps to observe flow behavior in many cases. During the initialization phase, the main steps are to define the flow properties at the beginning according to the physical simulation. Then, in the context of LBM, the discrete distribution function should also be initialized. In the compressible LBM solver, there are two discrete distributions to consider, namely f_i and g_i apart from the fluid properties. The most common initialization in the standard LBM scheme is to initialize the discrete distributions f_i and g_i with their respective equilibrium distributions f_i^{eq} and g_i^{eq}, as follows, [1].

$$f_i(x, t = 0) = f_i^{eq}(x, t = 0) \tag{3.42}$$

$$g_i(x, t = 0) = g_i^{eq}(x, t = 0) \tag{3.43}$$

The respective equilibrium distribution f^{eq} can be found using equation 3.29. However, to find g^{eq}, first, Lagrangian multipliers should be estimated using the Newton-Raphson scheme at $t = 0$. Thereafter, g^{eq} can be calculated according to equation 3.21.
It is worth noting that all the physical simulation parameters should be adopted into non-dimensional LBM units using a suitable conversion technique.

3.5.2 Boundary Conditions

Apart from the initialization, the definition of boundary conditions in LBM plays a pivotal role when mimicking real-world flow problems. Like in any other CFD simulation, the stability and accuracy of an LBM simulation mainly depend on the boundary conditions. Establishing boundary conditions in lattice Boltzmann simulations are more intricate than in conventional numerical solvers for the NSE. Instead of setting boundary values for macroscopic variables such as velocity or pressure, lattice Boltzmann simulations require us to define conditions for the mesoscopic populations. The primary challenge in this endeavor arises from the fact that the system of mesoscopic variables possesses a greater number of degrees of freedom compared to the corresponding macroscopic system. Specifically, in a particular lattice stencil, there are more discrete distributions to manage than macroscopic moments that need to be fulfilled. While acquiring the moments from the populations is a straightforward process, the reverse operation lacks uniqueness. Therefore, there are a variety of different boundary conditions in LBM [1]. In this context, there are three main boundary treatments applied in compressible LBM. They are the bounce-back method, periodic boundary conditions, first-order Neumann boundary conditions, and Dirichlet boundary conditions. When implementing the boundary conditions, the flag variable is used to define the nodes according to the boundary requirements. Since the present implementation uses the "pull" streaming method, an additional layer of boundary cells is equipped in the computational domain. Because of the standard lattice configurations used in the present case, one layer of additional helper lattice cells/nodes is enough outside the fluid domain, as shown in figure 3.2.

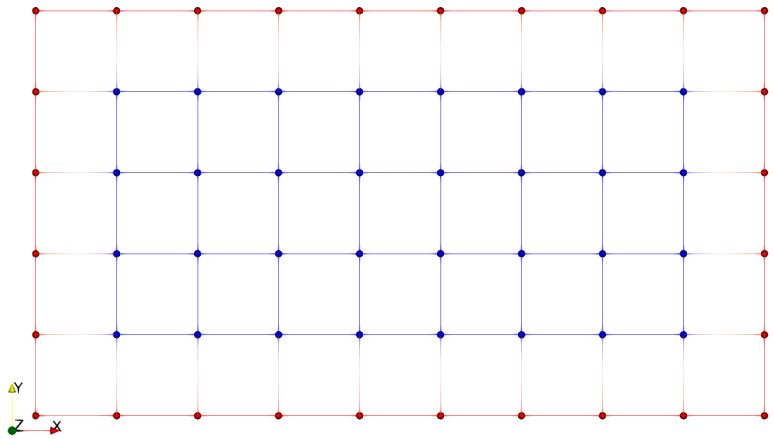


Figure 3.2: Definition of helper cells and fluid cells in a computational lattice domain

Figure 3.2, is an exemplary computational lattice, and the red lattice nodes represent the additional layer of helper nodes while the blue lattice nodes represent the fluid nodes. Thus, after collision in each time steps all the collided discrete distributions are written into the helper (red) nodes and thereafter, the standard "pull" streaming is carried out as stated previously. The values of discrete distributions are determined according to the choosen boundary conditions.

3.5.2.1 Bounce Back Boundary Scheme

The bounce-back boundary condition mimics the no-slip boundary condition in hydrodynamics. When it comes to modeling wall boundary conditions, the bounce-back method is the oldest lattice Boltzmann condition suitable for wall modeling. Therefore, it is the most commonly used boundary condition within the LBM community. In the bounce-back method,

the main principle is that the discrete distributions hitting a wall are reflected back to the lattice node where they originated along the respective discrete velocity link [1]. This can be illustrated in the following figure 3.3. The arrowheads represent the respective moving directions of the populations.
The dashed line between the blue lattice nodes and the red lattice nodes indicates the solid wall where red lattice nodes reside inside the wall. Therefore, in the bounce-back scheme, the wall is $\Delta x/2$ distance away from the nearby fluid lattice node.

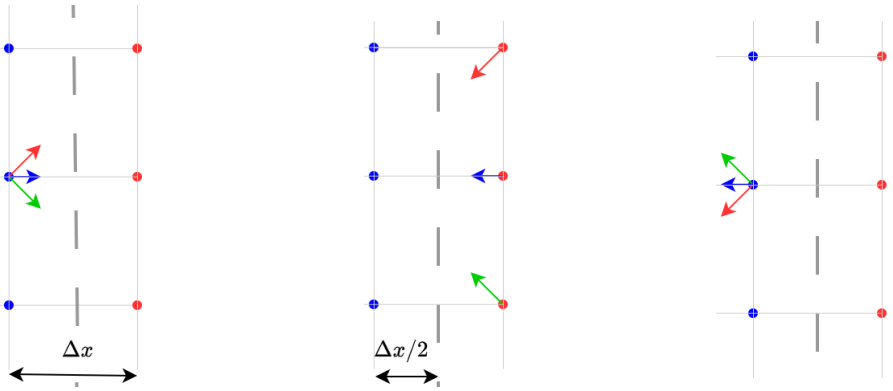


Figure 3.3: Bounce-back scheme for a solid wall. The population right after collision (a), assignment of reflected collided popultions to helper cells (b), streaming (c)

The bounce-back scheme is a simple, efficient, and stable boundary condition. Due to its simplicity, it can also be used in cases where the computational domain has a complex shape. However, the use of the simple bounce-back scheme in a complex simulation domain leads to the well-known effect called “staircase approximation” of the boundary. This can be visualized in figure 3.4.

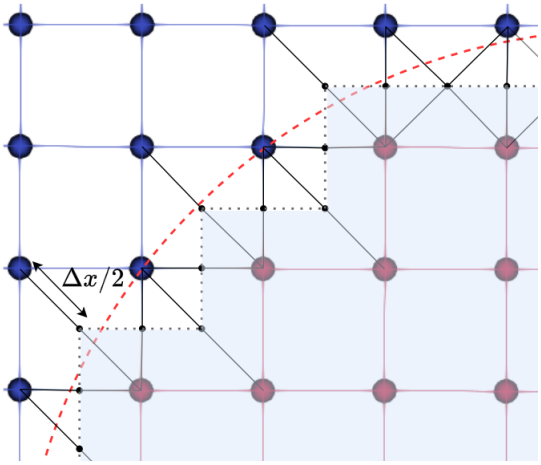


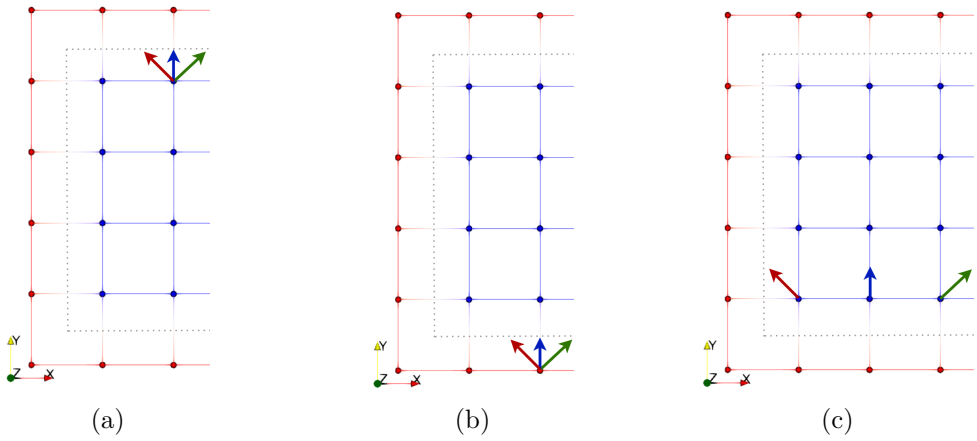
Figure 3.4: Staircase approximation of a circular boundary using simple bounce back method

In figure 3.4, the blue and red nodes represent the fluid boundary nodes and the boundary nodes on the solid wall. The small black dots represent the point $(\Delta x/2)$ exactly in between the connection line of two blue and red nodes. The red dashed line represents the circular wall, while the shaded area is the wall approximated by the simple bounce-back scheme. In general, the simple bounce-back method has greater accuracy than the first order if the surface

is aligned with the lattice nodes, and also the wall is in the center of the line connecting a fluid boundary node and a solid node. Apart from that, for any arbitrary shape, the simple bounce-back method is first-order accurate [1]. Therefore, to approximate arbitrary-shaped boundaries, one can also refer to different boundary treatments, such as the immersed boundary method, according to the preferred accuracy. However, in this thesis scope, the simple bounce-back scheme is used for all the no-slip boundary condition implementations.

3.5.2.2 Periodic Boundary Condition, First-Order Neumann Condition and Dirichlet Boundary Condition

Apart from the simple bounce-back scheme, the primarily interested boundary conditions in the present thesis scope are periodic boundary conditions, first-order Neumann conditions, and Dirichlet boundary conditions. In this section, a concise description of the above-mentioned boundary conditions will be provided.
In any numerical simulation scheme, the simplest boundary condition to implement is the periodic boundary condition. Periodic boundary conditions are applicable exclusively in scenarios characterized by periodic flow solutions. They specify that fluid exiting the domain from one side immediately re-enters from the opposite side. Consequently, these conditions maintain the conservation of mass and momentum throughout [1]. The periodic boundary scheme can be illustrated as shown in figure 3.5. However, it is always a good practice to check for the correct representation of physics when applying the periodic boundary condition.



(a) (b) (c)

Figure 3.5: Periodic boundary. The population right after collision (a), assignment of collided popultions to the bottom helper cells, that left the domain from top (b), streaming (c)

The other mainly used boundary condition within this compressible LBM implementation is the Neumann boundary condition. The main idea is that the normal derivative at a wall boundary is zero or a constant [23]. Therefore, this can be considered as $\partial f_i(x,t)/\partial \boldsymbol{x} = c$, where c is 0 or a constant. In the current implementations, the first-order Neumann condition is utilized in all the relevant simulations where necessary. As in many hydrodynamic simulations, the Dirichlet boundary condition is a common approach to specify the flow conditions at a boundary, specifically at inlets. The main idea in this boundary condition is to specify a fixed value at a relevant boundary [1]. Apart from the above-mentioned boundary treatments, the free-stream boundary condition is also applied in some simulations to recover the proper simulation physics.

3.6 Implementation of Root Finding Scheme

Since the compressible LBM in this thesis scope is based on entropic collision model, the equilibrium distribution g^{eq} needs to be estimated by using equation 3.21 which is restated below.

$$g_i^{eq} = \rho w_i(T)\ exp(\lambda + \mu_\alpha c_{i\alpha})$$

The Lagrangian multipliers λ and μ_α result from the minimization of the discrete entropy function 3.17. Therefore, to estimate the Lagrangian multipliers, the Newton-Raphson scheme is used in this implementation. In the Newton-Raphson scheme, the inverse of the Jacobian matrix is evaluated in each iteration. As discussed before, the size of the Jacobian matrix depends on the dimensionality. For instance, in a two-dimensional case, the Jacobian has a size of a 3×3 matrix, and in a three-dimensional case, the size is a 4×4 matrix. To accurately and efficiently find the inverse, one can implement an inverse finding algorithm from scratch. However, in practice, rather than implementing an inverse finding algorithm from scratch, it is better to incorporate existing libraries to find the inverse. In this thesis, for two-dimensional cases, an inverse finding algorithm is implemented from scratch to find the inverse of a 3×3 matrix by using the adjoint matrix and the determinant of the Jacobian. Since it is tedious and error-prone to have one's own inverse finder, for cases beyond 3×3 matrices, the GNU Scientific Library (GSL) is utilized to calculate the inverse. Therefore, in this compressible LBM implementation, for all three-dimensional flow problems, the inverse of the Jacobian matrix is calculated using GSL. (see Appendix: B.2).
With that, the Newton-Raphson root-finding scheme implemented in the present work can be stated in the following algorithm. It is important to notice that Jacobian matrices and other relevant matrices are treated as one-dimensional arrays in the implemented scheme. Furthermore, when time-stepping, only the fluid nodes are subjected to the root-finding scheme using the flag variable. Also, the tolerance value for the scheme is chosen carefully as 10^{-12}, which is slightly closer to machine precision. Finally, to calculate the Lagrangian multipliers at each lattice node, a one-dimensional array of the "grid" object is passed to the root-finding algorithm by reference. The implementation of the Newton-Raphson scheme implemented in the present work is stated in Algorithm 2.

Algorithm 2: Implementation of Newton-Raphson scheme

```
1  void newtonRaphson(grid& g,const double& cv,int flagIndex,double tol=1e^-12){
2  for (int i = 0; i < g.NZ * g.NY * g.NX; ++i)
3  {
4      if (g.domain[i].flag == flagIndex)
5      {
6          Initialize the required error margin as a double;
7          Initialize the the iteration counter to zero;
8          Set maximum number of iterations accordingly;
9          Initialize an 1-D array of doubles to zeros with element size equals to number
             of constraints for the function matrix;
10         Create an 1-D array of doubles with elements according to constraint size for
             the Jacobian matrix;
11         Create an 1-D array of doubles with elements according to constraint size for
             the inverse matrix;
12         while ((error > tolerance) && (iteration counter < maximum iterations))
13         {
14             Calculate the function values defined according to the constraints;
15             Calculate the inverse of Jacobian;
16             Calculate the residual matrix;
17             Update the function values according to x^{k+1} = x^k − δx where, δx being
                 the residual matrix;
18             Determine the convergence of the scheme according to the L_2 norm;
19             Update Lagrangian multipliers according to updated values;
20             Increase the iteration counter;
21         }
22     }
23 }
24 }
```

Chapter 4

Numerical Results; Validation and Verification

In this chapter, the implemented compressible LBM is verified and validated using several standard simulation cases. One of the main focuses of the thesis is to identify the speed of sound of a pressure wave. Therefore, this chapter will also describe the Riemann problem along with the shock tube simulations. Furthermore, this chapter will discuss the concept of shifted lattice stencils, which helps to extend the present implementation for supersonic flow simulations.

4.1 Numerical Shock Tube Experiments

Compared to incompressible fluids, the density of a fluid is a variable in a compressible medium. Furthermore, if the flow has sufficiently large velocities, the energy fluctuation within the fluid should also be considered in compressible fluids. This can be regarded as coupling between thermodynamics and aerodynamics. Moreover, shock waves can also appear in compressible fluids [24]. A shock wave is a sudden disturbance in a fluid that induces a discontinuous and irreversible change in fluid properties when fluid velocity turns from supersonic to subsonic. The main properties to consider in such scenarios are pressure, density, and fluid velocity. Due to temperature and velocity discontinuities, energy dissipation effects can occur in the fluid, making the fluid behavior thermodynamically irreversible. The thickness of a shock wave in an inviscid non-conducting gas is on the order of the molecular mean free path [11].

4.1.1 The Riemann Problem and the Shock Tube Problem

The Riemann problem is a special kind of initial value problem, illustrated by using the simplest form of a hyperbolic partial differential equation (PDE) with the following initial conditions. In this case, the initial discontinuity occurs at $x = 0$, and u represents the velocities.

$$\text{PDE} : \frac{\partial u}{\partial t} + \frac{\partial x}{\partial t}\frac{\partial u}{\partial x} = 0 \tag{4.1}$$

$$\text{Initial conditions} : u(x,0) = u_0(x) = \begin{cases} u_{Left} & \text{if } x < 0 \\ u_{Right} & \text{if } x > 0 \end{cases} \tag{4.2}$$

The solution for a Riemann problem consists of a mathematical and physical description of relevant conservation laws. The exact solution of Riemann problems has been used as an

evaluation method for the performance and accuracy of a particular numerical scheme when developing compressible fluid flow solvers. Considering gas dynamics, the Euler equations, and the shock-tube problem, which can be seen as a special case of the Riemann problem where the initial velocities are equal to zero [25].
The shock tube serves as an experimental device for studying short-term dynamics in unsteady compressible fluid phenomena. Different wave structures and wave interactions of shock waves can be observed in shock tube experiments using optical methods or high-speed photography techniques. As illustrated in figure 2.5, the shock tube consists of two sections separated by a diaphragm. One side contains gas with higher pressure, known as the driver section, while the other side has low-pressure gas, referred to as the expansion section or driven section. Initially, a normal shock wave is generated by rupturing the diaphragm in the middle of the two sections, and the propagation of wave structures are observed. The typical wave patterns inside a shock tube are shown in figure 4.1 [11].

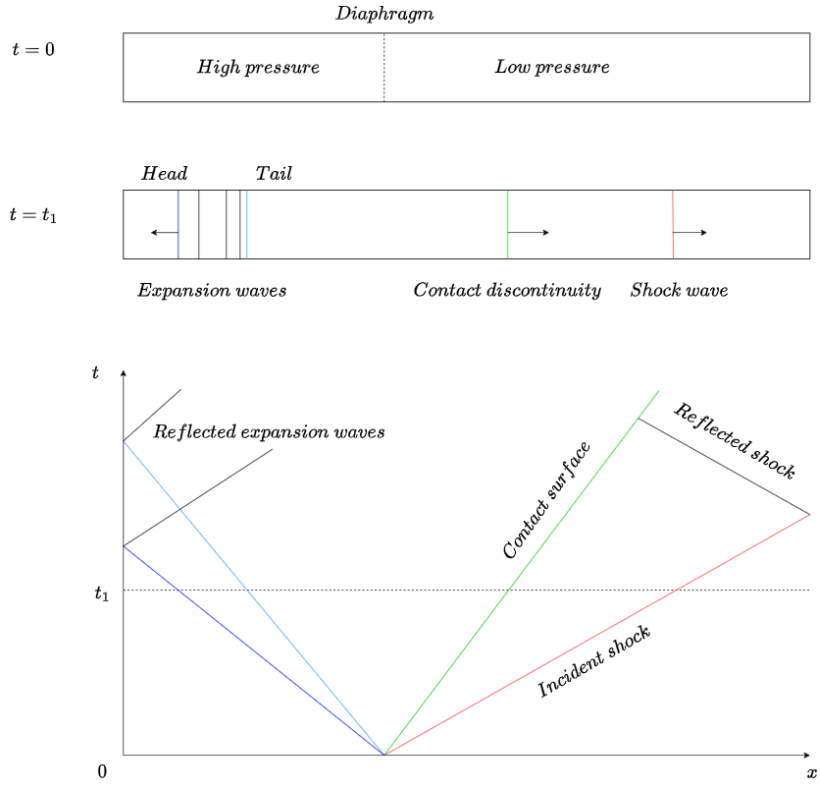


Figure 4.1: Wave characteristics inside a shock tube, and the relevent characteristic lines for different wave fronts represented in $x - t$ diagram

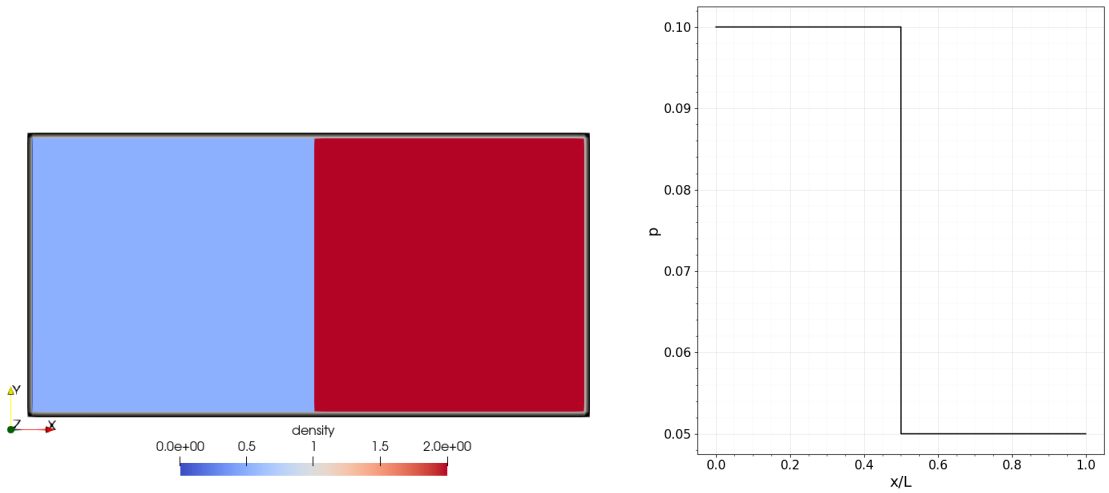
Initially, the diaphragm is abruptly broken, and due to the pressure difference, a shock wave is propagated to the lower pressure side. A schematic representation of wave configurations at time $t = t_1$ is also depicted in figure 4.1. A series of expansion waves travel in the opposite direction to the shock wave. Inside the shock tube, the velocity between the tail of the expansion wave and the shock wave is equal. However, a contact discontinuity can be observed due to the change in temperature and entropy. The wave propagation speed of the shock wave is the velocity mentioned above, between the shock wave front and the tail of the expansion wave [11]. The characteristic lines shown in the $x - t$ diagram have a gradient equal to the velocity corresponding to each wave front. This structure resembles the solution of the Riemann problem [25].

4.1.2 Implementation of Shock Tube Using Compressible LBM

In this section, the numerical investigation of shock tube phenomena is conducted using the implemented compressible lattice Boltzmann method. Considering the propagation of normal shock waves inside the shock tube, all relevant simulations are carried out as two-dimensional simulations. Therefore, the D2Q9 standard lattice configuration is employed for the compressible LBM implementation. As a preliminary study, a shock tube with a small pressure difference is numerically investigated. For this study, the fluid properties and the dimensions of the interested fluid domain are based on [10].
Initially, properties such as density, temperature, and fluid velocity inside the shock tube are stated in Table 4.1. All the properties are in lattice units. The length of the shock tube is 3000 lattice units in the x-direction and 4 in the y-direction. Due to the D2Q9 standard lattice, only one layer of helper lattice nodes is used for handling the boundary conditions. Equal lattice spacing of $\Delta x = \Delta y = 1$ is utilized, while the time step is $\Delta t = 1$. The initial discontinuity of the shock tube is at the middle of the lattice nodes in the x-direction. The kinematic viscosity, Prandtl number (Pr), and the specific heat ratios of the fluid are taken according to [10] as 0.025, 0.71, and 1.4, respectively. For the verification of the implemented compressible LBM, a total simulation time of 1273 is adopted accordingly.
The results obtained using the compressible LBM implementation are then compared with the exact shock tube results using the Fortran code provided in [26]. As mentioned previously, considering the Euler equations, the shock tube problem is a special case of the Riemann problem. However, it is not an exact closed-form solution to the Riemann problem for the Euler equations. These exact solutions are based on iterative schemes [25].

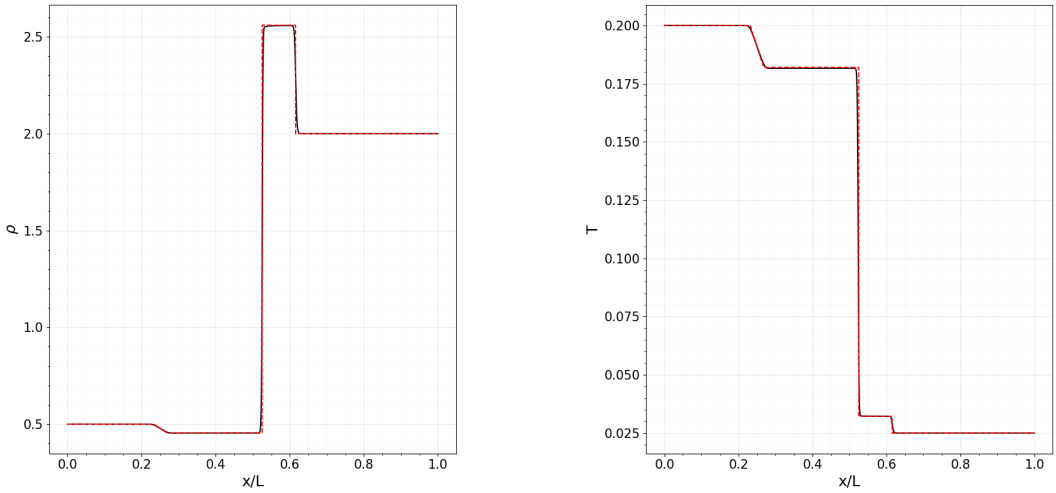
	Left $(x/L_x < 0.5)$	**Right** $(x/L_x \geq 0.5)$
density (ρ)	0.5	2.0
velocity (u_x, u_y)	(0,0)	(0,0)
Temperature (T)	0.2	0.025

Table 4.1: Initial macroscopic properties inside the shock tube with small pressure difference



(a) (b)

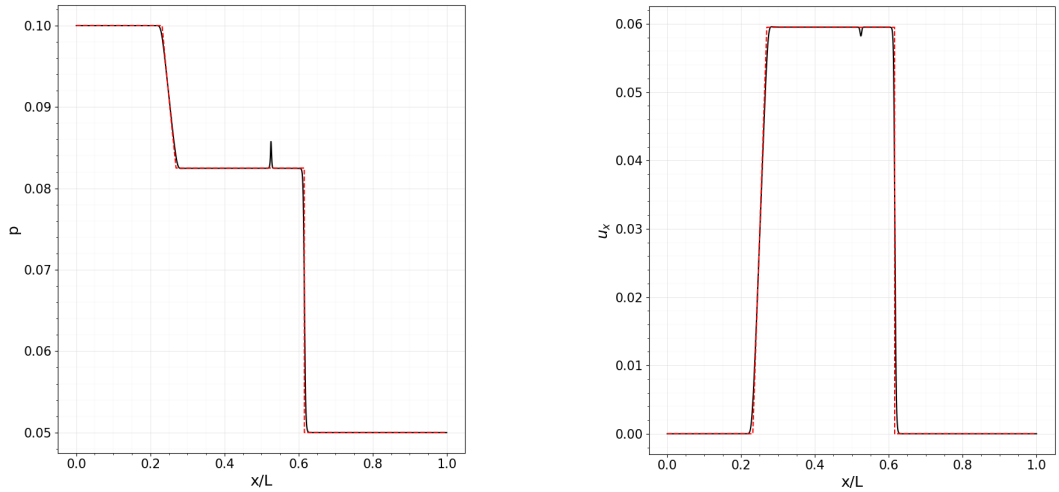
Figure 4.2: (a) Contour plot of initial density variation inside the domain (not in scale) and (b) initial pressure variation inside the domain against the normalized length



(a) Density variation inside the shock tube against normalized shock tube length

(b) Temperature variation inside the shock tube against normalized shock tube length

Figure 4.3: Comparison of exact shock tube results [26] with compressible LBM implementation



(a) Pressure variation inside the shock tube against normalized shock tube length

(b) x-velocity variation inside the shock tube against normalized shock tube length

Figure 4.4: Comparison of exact shock tube results [26] with compressible LBM implementation

In the above figures 4.3 and 4.4, the dashed red line represents the results for exact values, while the black line represents the corresponding results for the compressible LBM implementation. The compressible LBM matches well with the exact results apart from a spike in figure 4.4. Further, as mentioned in section 4.1.1, the characteristic of the contact discontinuity can also be realized by comparing 4.4b with the rest of the figures. This is because, even though the velocity has a constant value approximately between $x/L \in$ [0.2,0.6] the other macroscopic properties change due to the increase in temperature and entropy. The maximum velocity in 4.4b is the propagation speed of the shock wave, and it is similar to the speed of sound in this context. In compressible LBM, the definition for finding the Mach number is stated as follows in equation 4.3 according to [8][11].

$$Ma = u/\sqrt{\gamma T} \tag{4.3}$$

Considering the above definition, a comparison of exact Mach number with present compressible LBM is shown in figure 4.5.

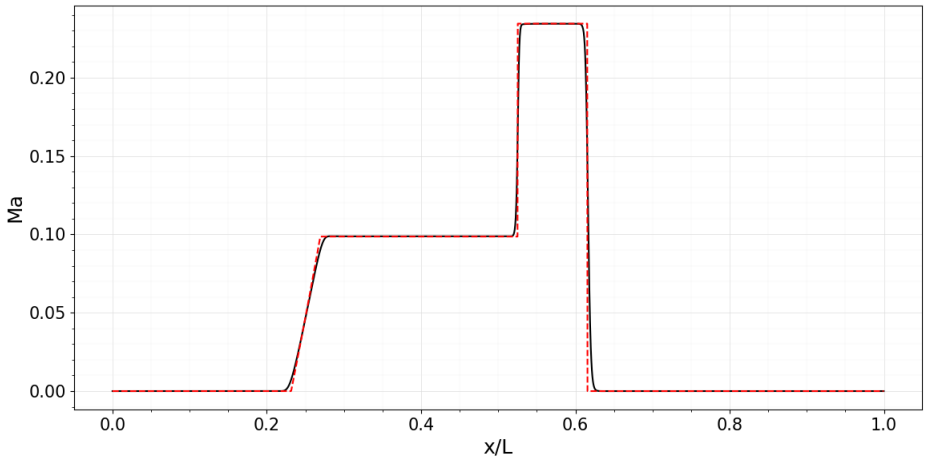


Figure 4.5: Comparison of exact and compressible LBM Mach number profiles against normalized shock tube length

According to figure 4.5, the maximum Mach number in this case is approximately 0.235. The size of the Mach number can be increased by increasing the pressure difference between the two sides [27]. The Sod shock tube problem [28] stands as a canonical benchmark for assessing the capabilities of compressible solvers [8], where the initial pressure ratio is 10. Thus, a Sod shock tube case is investigated below using present compressible LBM implementation. The initial conditions for the Sod shock tube case are stated in Table 4.2, as depicted in [9].

	Left $(x/L_x < 0.5)$	**Right** $(x/L_x \geq 0.5)$
density (ρ)	1.0	0.125
velocity (u_x, u_y)	(0,0)	(0,0)
Pressure (P)	0.15	0.015

Table 4.2: Initial macroscopic properties for Sod shock tube problem

The computational domain has 600 lattice nodes in the x-direction and 4 lattice nodes in the y-direction. The dynamic viscosity, Prandtl number, and specific heat ratio are 0.015, 0.71, and 1.4, respectively. The macroscopic properties after $t = 310$ are visualized and compared with the exact results.

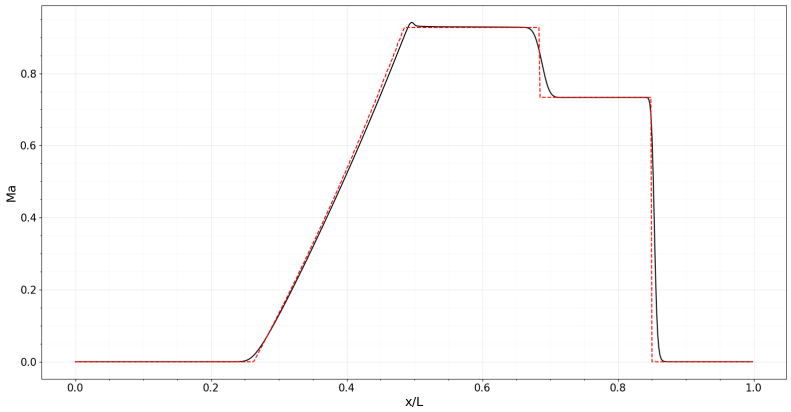
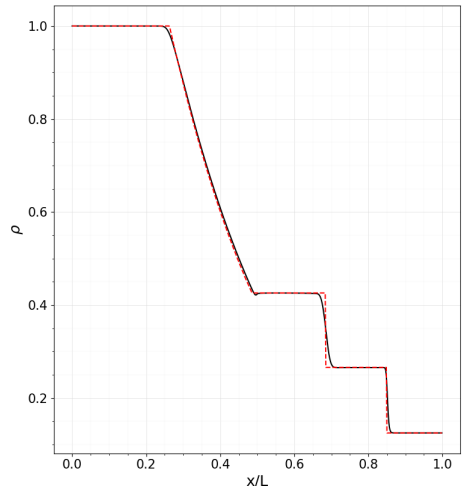
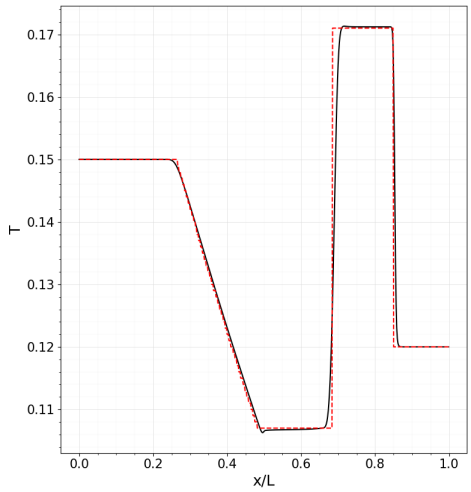


Figure 4.6: Comparison of exact and compressible LBM Mach number profiles against normalized shock tube length for Sod shock tube problem

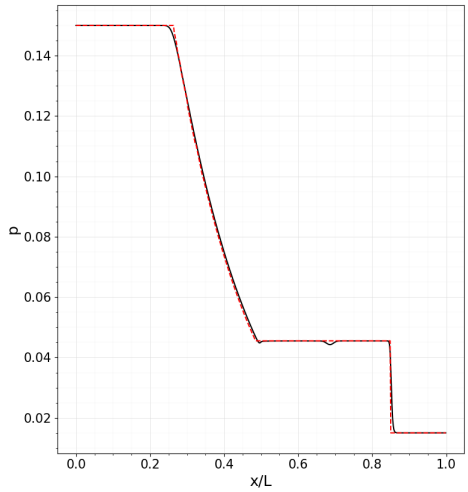


(a) Density variation inside the shock tube against normalized shock tube length

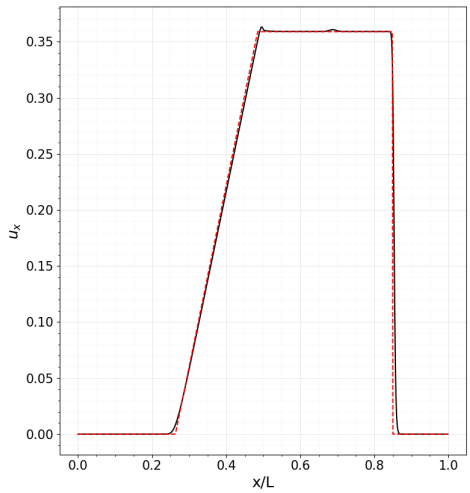


(b) Temperature variation inside the shock tube against normalized shock tube length

Figure 4.7: Comparison of exact shock tube results [26] with compressible LBM implementation for Sod shock tube problem



(a) Pressure variation inside the shock tube against normalized shock tube length

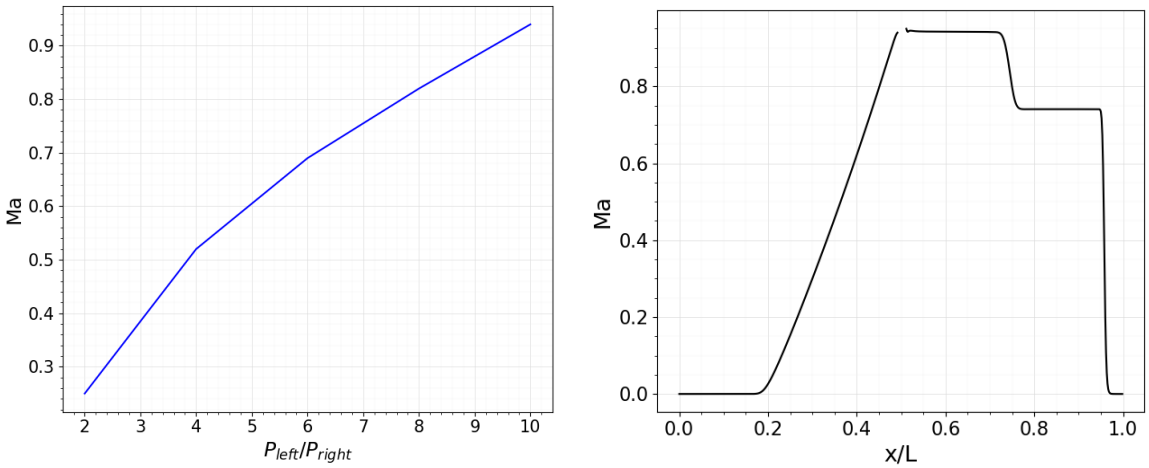


(b) x-velocity variation inside the shock tube against normalized shock tube length

Figure 4.8: Comparison of exact shock tube results [26] with compressible LBM implementation for Sod shock tube problem

As shown in figure 4.6, the attainable maximum Mach number is approximately 0.9 for the Sod shock tube problem. To further investigate the limits of the present implementation, the pressure difference across the diaphragm is changed to increase the Mach number. The pressure ratio ($P_{\text{left}}/P_{\text{right}}$) versus the maximum attainable Mach number is plotted in figure 4.9a. According to the investigation, the Mach number reaches a limit of approximately 0.95. Moreover, a pressure ratio beyond 10.26 provides a singular matrix when approximating the g_i^{eq}. Furthermore, the discontinuities inside the shock tube due to the strong shock formations cannot be handled by the implementation, as shown in figure 4.9b. Therefore, with Algorithm 1, the compressible LBM lies in the subsonic flow regime. Hence, a couple of techniques should be incorporated into the compressible LBM implementation to extend the solver be-

yond a Mach number of 1.0. This will be discussed in the next section.



(a) Mach number inside the shock tube against pressure ratio

(b) Mach number inside the shock tube against normalized shock tube length

Figure 4.9: Saturating trend of Mach number with pressure ratio (a) and the inability to handle the discontinuities inside the domain (b).

Furthermore, it is intuitive that Algorithm 1 has additional complexities compared to the general standard LBM algorithm. Especially, due to the use of the root-finding algorithm, it will also increase the computational cost. Therefore, to investigate the number of Newton-Raphson iterations when estimating the g^{eq}, the number of Newton-Raphson iterations along the shock tube length is plotted, as shown in figure 4.10, considering the above Sod shock tube benchmark.

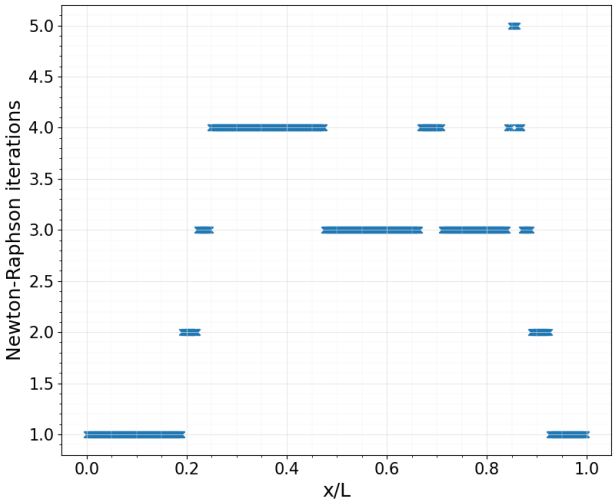


Figure 4.10: Newton-Raphson iterations against the normalized shock tube length

According to the observation, in the present implementation, it is evident that Algorithm 1 requires five iterations when resolving strong shock fronts. Therefore, stable and accurate results can be obtained by using a relatively smaller number of iterations. This will be further

observed in later benchmarks.

4.2 Shifted Lattice Scheme for Standard Lattice Configurations

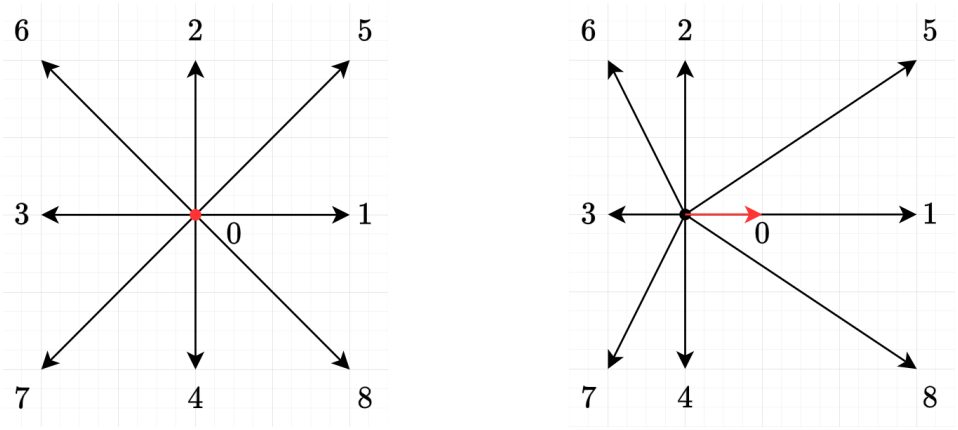
The standard LBM is recognized for its tendency to violate Galilean invariance, especially when it approaches the speed of sound c_s. Galilean invariance asserts that physical phenomena unfold uniformly in all inertial systems, that is in systems moving at constant velocities relative to each other. However, the physics of flow problems in nature is independent of the reference frame [1].

4.2.1 Shifted Lattice Velocity

In the context of both isothermal and compressible models, it is noted that discrepancies in higher-order moments exhibit a scaling relationship with the divergences of local temperature and velocity from their respective lattice reference values. One of the reasons for this violation is the use of symmetric lattice configurations that consider the initial reference frame at rest. Additionally, to overcome this issue in higher-order Mach numbers, a lattice stencil with larger velocity sets should be adopted [29].
Therefore, in shifted lattice schemes, equilibrium is considered in a moving reference frame with a non-zero velocity shift U in the flow direction. The shifted discrete velocity c' can be obtained according to equation 4.4.

$$c'_{i\alpha} = c_{i\alpha} + U_\alpha \tag{4.4}$$

As a result of shifted velocities, the stencil is no longer symmetric. Furthermore, the Eulerian nature of the standard stream and collide algorithm is affected by the shift velocity. This is because now the discrete distributions move with the additional non-zero discrete velocity. However, while the shift can be an integer for stencils with extended neighborhoods, for standard stencils, the shift velocity should be less than 1.0 due to stability issues [17]. Figure 4.11 indicates the effect of shifted velocity for a D2Q9 standard stencil.



(a) $U = (0.0, 0.0)$ (b) $U = (0.5, 0.0)$

Figure 4.11: D2Q9 stencil with shift velocity $U = (0.0, 0.0)$ (a) and shift velocity $U = (0.5, 0.0)$ (b)

Interestingly, the temperature-dependent weights are Galilean invariant for any set of discrete velocities, irrespective of the shift velocity, and equilibrium distributions are in the same form.

Since equilibrium is now performed in a moving frame, the shifted discrete velocity is utilized when approximating the equilibrium distributions [29]. Therefore, when approximating the discrete equilibrium distribution f^{eq}, the equations 4.5 – 4.8 can be adopted according to Saadat et al. [8]. A elaborated example of f_i^{eq} considering D2Q9 stencil is stated in Appendix A.2

$$f_i^{eq} = \rho\ \phi_{ix} \cdot \phi_{iy} \cdot \phi_{iz} \tag{4.5}$$

where,

$$\phi_{(0)} = 1 - ((u_\alpha - U_\alpha)^2 + T) \tag{4.6}$$

$$\phi_{(+1)} = \frac{(u_\alpha - U_\alpha) + (u_\alpha - U_\alpha)^2 + T}{2} \tag{4.7}$$

$$\phi_{(-1)} = \frac{-(u_\alpha - U_\alpha) + (u_\alpha - U_\alpha)^2 + T}{2} \tag{4.8}$$

Considering the discrete equilibrium distribution g^{eq}, the constraint 3.19 should be incorporated with the shifted lattice stencil as in equation 4.9 when minimizing the discrete entropy function 3.17. Then, g^{eq} takes the form in equation 4.10.

$$q_\alpha^{eq} = \sum_{i=0}^{Q-1} c'_{i\alpha} g_i^{eq} = 2\rho u_\alpha (E + T) \tag{4.9}$$

$$g_i^{eq} = \rho w_i(T)\ exp(\lambda + \mu_\alpha c'_{i\alpha}) \tag{4.10}$$

4.2.2 Inverse Distance Weighted Interpolation.

As can be observed in figure 4.11, the lattice is no longer a space-filling lattice while being non-symmetric. However, if the shift is an integer, the resulting lattice is space-filling. Therefore, only the streaming step of the standard LBM algorithm is adopted with discrete shift velocity [29]. Due to the non-integer shift in standard stencils, there should be an interpolation/extrapolation scheme of the discrete equilibrium distribution before the streaming step. Initially, a semi-Lagrangian scheme is proposed by [8]. However, in the present implementation, this provided an overfit while reconstructing the distribution at lattice nodes at rest. Therefore, the Inverse Distance Weighted interpolation (IDW) is adopted for the current implementation [10].
IDW interpolation constructs the values of unknown data points by weighting the average of known points. The standard form of the IDW method is shown in equation 4.11 [30].

$$Z(x) = \sum_{i=1}^{n} \frac{\omega_i(x) z_i}{\sum_{j=1}^{n} \omega_j(x)'} \tag{4.11}$$

where,

$$\omega_i(x) = \frac{1}{d(x, x_i)^\alpha} \tag{4.12}$$

In the above formulas 4.11 and 4.12, $Z(x)$ is the interpolated value at position x, $\omega(x)$ is the weighting function, where $d(x, x_i)$ is the distance between the known point and the unknown point. α is the distance decay parameter, conventionally taken as 2.0. The variable n represents the sample point size [30].
In the compressible LBM algorithm, the IDW interpolation step is performed before the streaming step. The following figure 4.12 shows a computational domain at a particular time

instance of the compressible LBM stencil with a shift velocity of U_{shift}. The interpolated discrete distributions are denoted as f_i^-.

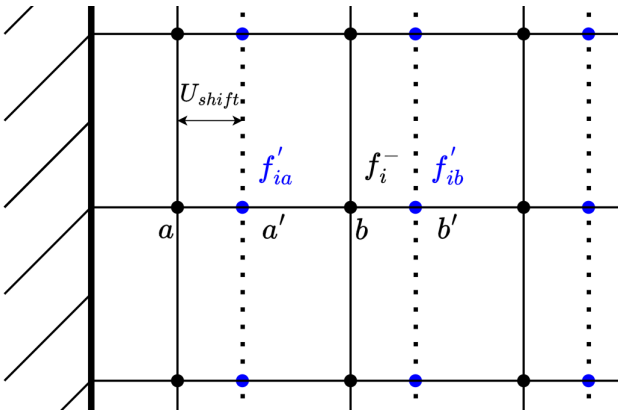


Figure 4.12: Moving frame (blue) and stationary frame (black) at a particular time step with a $0 < U_{shift} < 0.5$

Considering the nodes a', b, b' the interpolated discrete distribution f_i^- at b is approximated according to the IDW definitions as stated in equations 4.11 and 4.12 as follows,

$$f_i^- = f'_{ib} \cdot U_{shift} + f'_{ia} \cdot (1.0 - U_{shift}) \tag{4.13}$$

In equation 4.13, the decaying parameter α is taken as 1.0 according to [10]. Furthermore, to construct the discrete distribution functions at node a, extrapolation should be adopted accordingly.

4.3 Improvement in Stabilty for Supersonic Flows

In the supersonic flow regime, fluids exhibit strong compressibility effects. Even though the entropic LBM has better stability compared to other compressible models, it is not sufficient when handling supersonic flows. Furthermore, special consideration should be given to handling low viscosities and under-resolved conditions. Considering this, Latt et al. [6] proposed a stability technique based on the Knudsen number (Kn). The Knudsen number is a non-dimensional parameter that characterizes continuum flows and rarefied flows. It is defined as the ratio between the mean free path of gas molecules and the characteristic length of the flow scenario. According to the Knudsen number, a flow is considered continuum when $Kn < 0.01$. Rarefied gas flows are considered as flows with $Kn \geq 1$ [11].
The improved stability technique in the present compressible LBM solver is mainly an adaptation of the work from [6]. Thus, a concise description of the stability technique proposed in [6] is briefly discussed here. Based on that, the relaxation time τ is modified at each time step in every lattice node according to the following formula 4.14.

$$\epsilon = \frac{1}{Q} \sum_{i=0}^{Q-1} \frac{|f_i - f_i^{eq}|}{f_i^{eq}} \tag{4.14}$$

The Chapman-Enskog analysis shows that $\epsilon \approx Kn$ considering the Navier-Stokes Fourier equations. Therefore, depending on the continuum and rarefied regimes, the temperature-dependent τ is modified by using a piecewise linear function α as in equation 4.15. As a result,

spurious oscillations are dampened in under-resolved areas in the computational domain.

$$\alpha = \begin{cases} 1.0, & \epsilon < 0.01 \\ 1.05, & 0.01 \leq \epsilon < 0.10 \\ 1.35, & 0.10 \leq \epsilon < 1.0 \\ 1/\tau, & \epsilon \geq 1.0 \end{cases} \tag{4.15}$$

Then, based on the value of α, the required τ can be approximated by using equation 4.16.

$$\tau = \alpha\tau \tag{4.16}$$

Furthermore, this kinematic stabilization technique can be integrated into the algorithm without breaking the parallelizing capabilities of the overall compressible LBM algorithm. However, as mentioned at the beginning of this section, this stability technique should be applied in the supersonic flow regime. Therefore, the Kn number-dependent stabilization is case-dependent and should be verified with the correct physics. For a detailed description of this technique, interested reader may refer to the original publications at [6] and [17]. The final version of the compressible LBM algorithm with the above-mentioned improvements can be stated as in Algorithm 3.

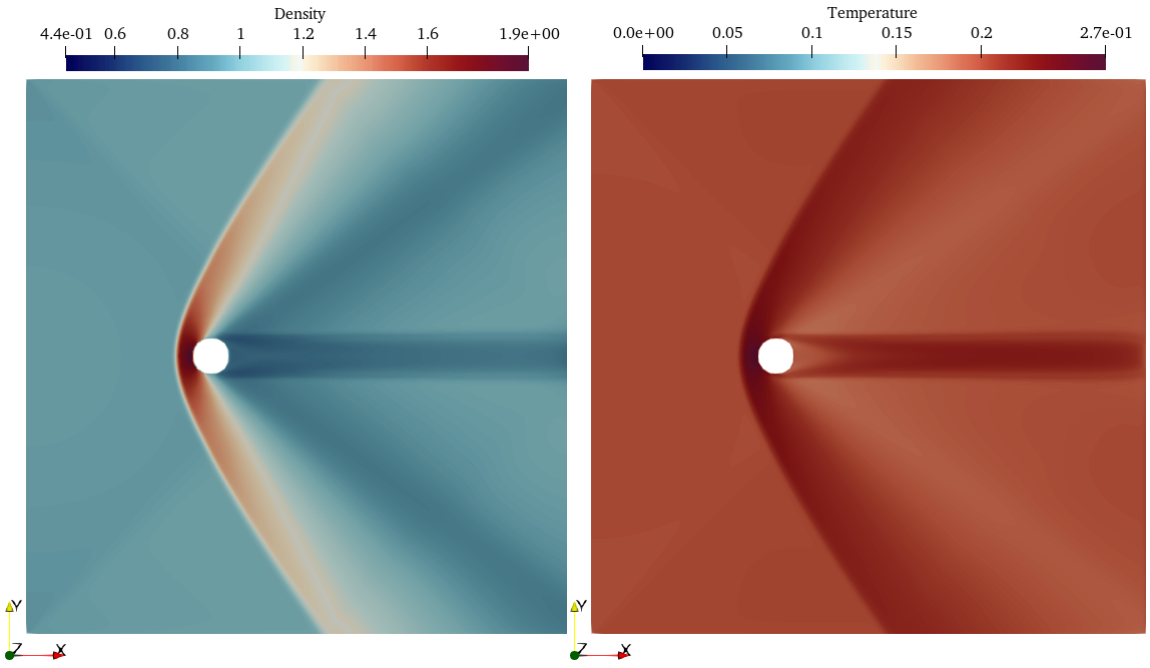
Algorithm 3: Compressible LBM algorithm for strong compressible fluid flows.

Data: Read computational grid and fluid properties;
Initialization: $\rho, \mathbf{u}, T$ and set temperature dependent weights;
Modification of c_i according to the shifted lattice velocity U;
Calculate: f^{eq};
Estimate Lagrangian multipliers $\leftarrow$ Newton-Raphson ;
Calculate: g^{eq} and Initialize g, f;
for $(int\ i = 0; i < Number\ of\ time\ steps; ++i)$
{
    Calculate $\rho, \mathbf{u}, T$ and set temperature dependent weights;
    Calculate time dependent relaxation times;
    Write .vtk files for post-processing;
    Calculate f^{eq};
    Estimate Lagrangian multipliers $\leftarrow$ Newton-Raphson;
    Calculate g^{eq};
    Calculate pressure tensor;
    Calculate quasi-equilibrium distribution g^*;
    Applying Knudsen number dependent stabilization;
    Collide;
    Reconstruction of f, g at grid nodes;
    Set boundary conditions;
    Stream;
    Swap f^{new} and f^{old};
}

4.4 Compressible Flow Over Two-Dimensional Sphere

The flow over a cylinder is extensively studied through numerical simulations and experimental techniques. These studies help to validate theoretical models, understand the complex

fluid dynamics, and provide insights into the behavior of flows around bluff bodies. In this section, compressible flow over a cylinder is investigated to verify and validate the compressible LBM implementation.
The square two-dimensional computational domain is simulated with 600 lattice nodes in each x and y axis. The choice of dimensions is based on [10] to maintain the consistency of results for verification purposes. The cylinder center is placed at the coordinate position (200,300) inside the computational domain. The cylinder has a diameter of 40 in lattice units. Being consistent with the standard LBM, the space and time discretization are performed as $\Delta x = \Delta y = \Delta t = 1$. Considering the boundary conditions, the inlet has Dirichlet boundary conditions, and the outlet is modeled using first-order Neumann conditions. The free stream boundary condition is applied to the top and bottom boundaries. The no-slip condition for the cylinder wall is implemented using the simple bounce-back scheme.
At the beginning of the simulation, the free stream conditions for density and temperature are 1.0 and 0.25, respectively. The specific heat ratio is 1.4, and the Prandtl number of the problem is 0.71. The investigated Reynolds number of the flow problem is 300. Since the margin of interest is beyond the Mach number of 1.0, the shifted lattice scheme is integrated with the Knudsen number-dependent stabilization. The shifted lattice velocity vector of $U = (0.5u_\infty, 0)$ is applied. To calculate u_∞, equation 4.3 is utilized. The capabilities of the present implementation are investigated in the transonic and supersonic regime, and the critical properties such as density, temperature, and the Mach number are visualized in the following figures with the help of ParaView.



(a) Density (b) Temperature

Figure 4.13: Density and temperature variation around 2-D cylinder for Mach number = 1.4 and Re = 300

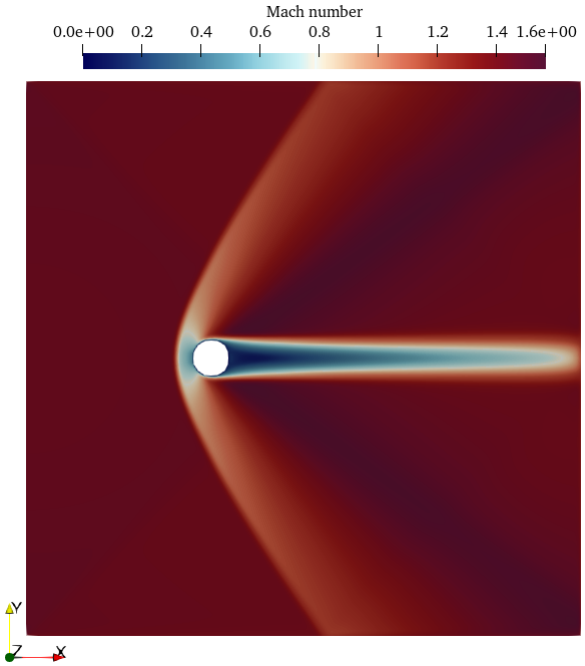


Figure 4.14: Mach number variation around 2-D cylinder for Mach number = 1.4 and Re = 300

According to figure 4.13a and 4.13b, the formation of a bow shock in front of the cylinder can be investigated, as expected for high Mach number flows. Further, this can also be visualized in figure 4.14. The obtained results for shock stand-off distance are compared with experimental, analytical, and simulated data for the validation and verification. The distance from the cylinder front to the bow shock front is measured along the stagnation streamline, and then that distance is normalized using the cylinder diameter to obtain the shock stand-off distance. Figure 4.15 shows the present compressible LBM results compared to experimental, analytical, and simulated results as in [6]. The circular blue dots represent the results from the present implementation.
According to the results, it is evident that the simulated results closely follow the trend stated in the literature. The dashed line represents the normalized shock stand-off distance described by Ambrosio and Wortman [31], while the other results indicate the experimental results except for the LBM results obtained by D3Q39, DDF-LBM approach according to Latt et al. [6]. Interestingly, the current LBM approach shows descent results even though it is based on the D2Q9 approach compared to DDF-LBM. Especially in the supersonic flow regime, it shows better results compared to the transonic flow regime. Further, it is also worth noting that the highest reachable Mach number indicated in the recent literature [10] is 1.8 and in this work the simulations are carried out upto the Mach number value of 2.0.

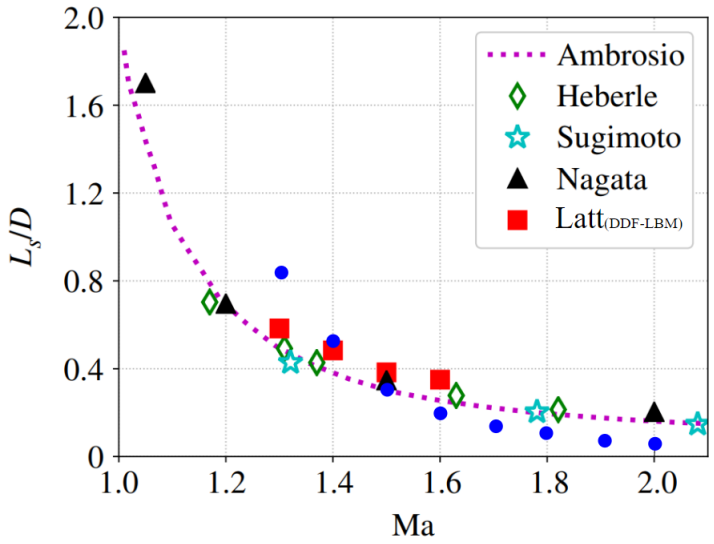
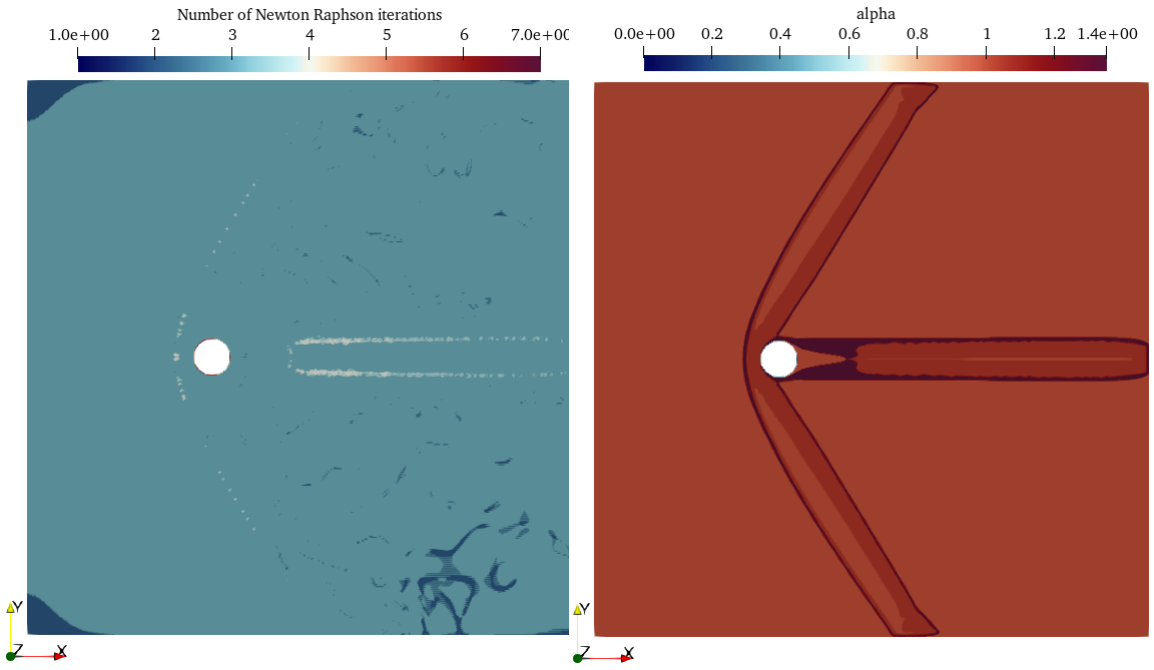


Figure 4.15: Comparison of obtained shock stand-off distance (blue points) with experimental and numerical data with increasing Mach number.

Furthermore, as mentioned in the shock tube case, the number of Newton-Raphson iterations in this benchmarking problem is also less than 10 iterations. This is visualized in figure 4.16a. According to the observations, mostly the iteration count stays around 3 – 5 iterations. As in the shock tube case, a higher iteration count is required to resolve strong shock formations. Moreover, the difference between the continuum and rarefied regimes can be observed due to the applicability of the Knudsen number-dependent stability scheme, as shown in figure 4.16b. The line of discontinuity of the flow domain because of the strong shock can be clearly visualized in figure 4.16b.



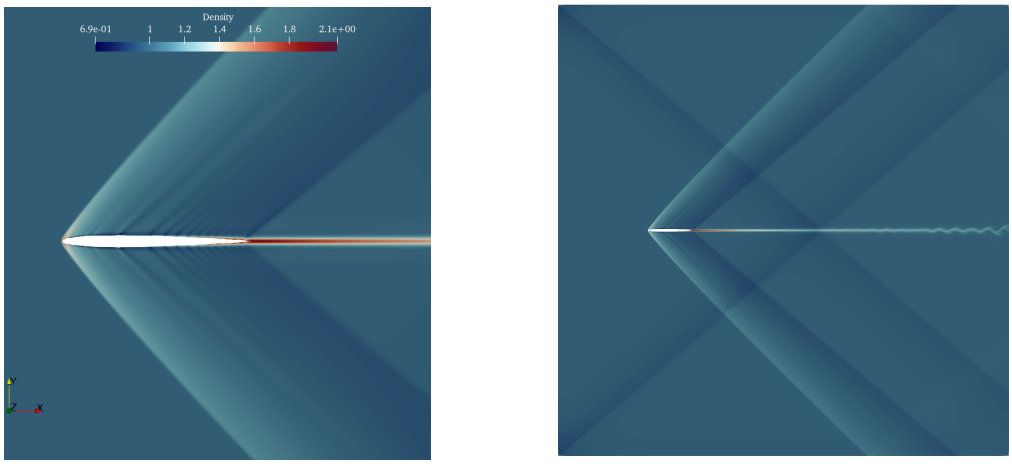
(a) Newton Raphson iterations

(b) α variation

Figure 4.16: Number of Newton Raphson iterations and alpha variation around 2-D cylinder for Mach number = 1.4 and Re = 300

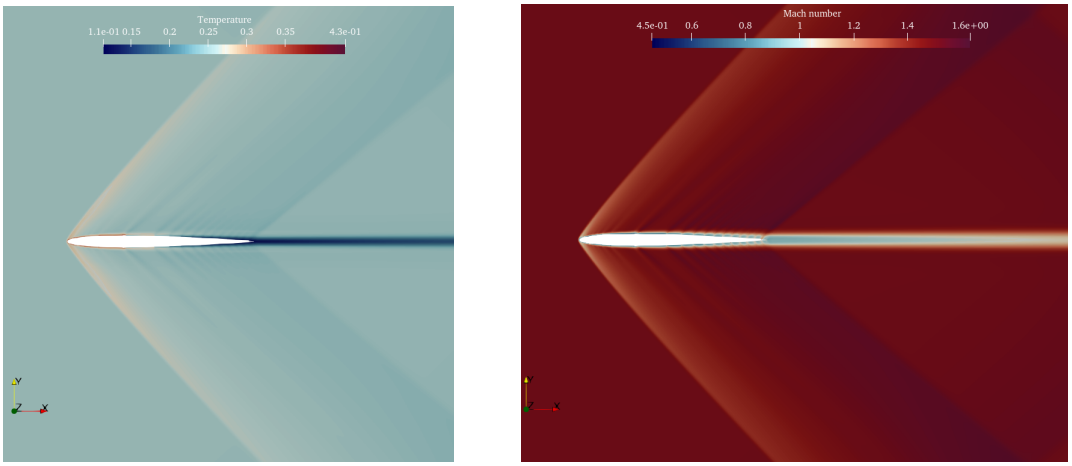
4.5 Compressible Flow Over Two-Dimensional NACA0012 Airfoil

In this section, the characteristics of the NACA0012 airfoil are investigated using compressible LBM implementation for its validation. The interested two-dimensional computational domain has 4000 lattice nodes in the x-axis and 4000 lattice nodes in the y-axis. The airfoil chord length is discretized using 400 lattice units. Initially, the leading edge of the airfoil is placed at coordinates (800, 2000) with a zero angle. The initial Mach number and the Reynolds number are selected according to the literature [29], [6] and [10] to observe the consistency of the results. Therefore, in this case the Mach number, Ma = 1.5, and the Reynolds number, Re = 10^4. Now the Reynolds number depends on the number of lattice units along the chord length. As in the previous flow over 2-D cylinder benchmark, the inlet boundary condition is modeled using Dirichlet boundary condition, while the outlet is modeled using first-order Neumann condition. The top and bottom boundaries are modeled using free stream boundary conditions. The airfoil surface is implemented using a no-slip condition with a simple bounce-back scheme.
Initially, the free stream density and velocity of the domain are set at 1.0 and 0.25, respectively. Furthermore, the specific heat ratio, $\gamma = 1.4$, and Prandtl number, $Pr = 0.71$ are utilized, while the shifted lattice velocity U is set as $(0.5u_\infty, 0.0)$. The obtained results for the density, temperature, and Mach number profiles around the airfoil are shown in the following figures.



(a) Density (b) vortex formation

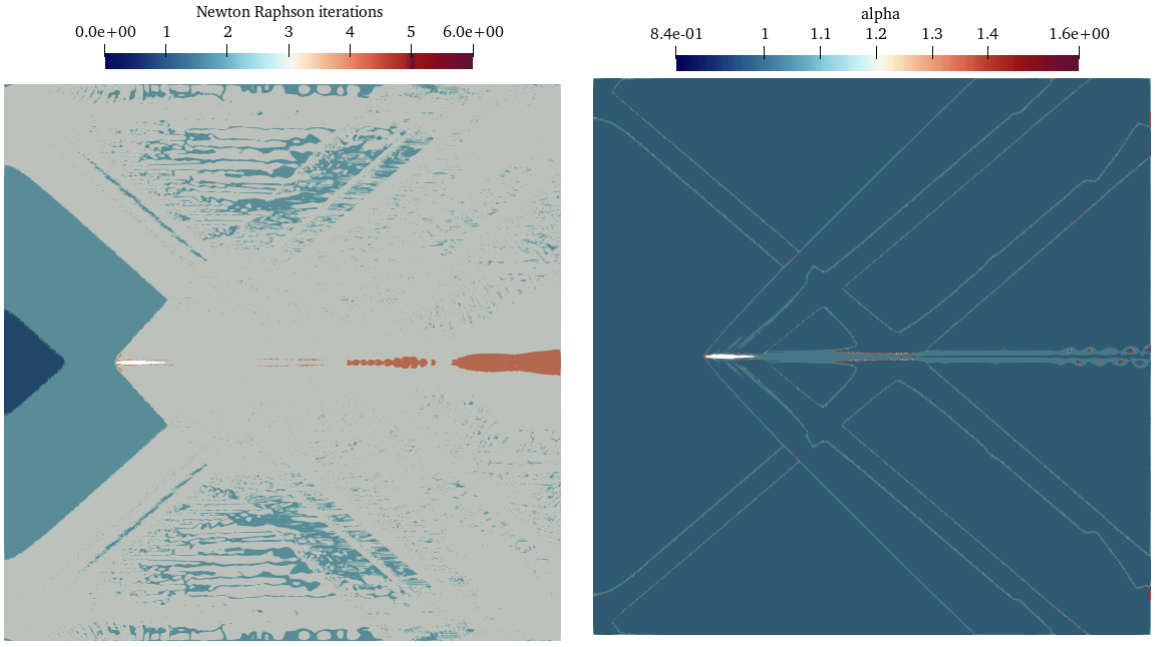
Figure 4.17: Density variation around airfoil for Mach number = 1.5 and Re = 10^4 and the downstream vortex formation



(a) Temperature (b) Mach number

Figure 4.18: Temperature and Mach number variation around airfoil for Mach number = 1.5 and Re = 10^4

According to the observations, it can be seen that the density and temperature increase at the leading edge of the airfoil due to the strong shock wave formation. Further, a weak shock wave can also be observed near the trailing edge of the airfoil, as predicted in previous literature [29], [6]. As depicted in figure 4.17b, the shedding of vortices in the downstream flow field is also captured by the present compressible LBM implementation, even though it is not equipped with any grid refinement techniques. To understand the numerical effort of the solver, the number of Newton-Raphson iterations and the variation of the α function are also investigated, as depicted in the following figures.



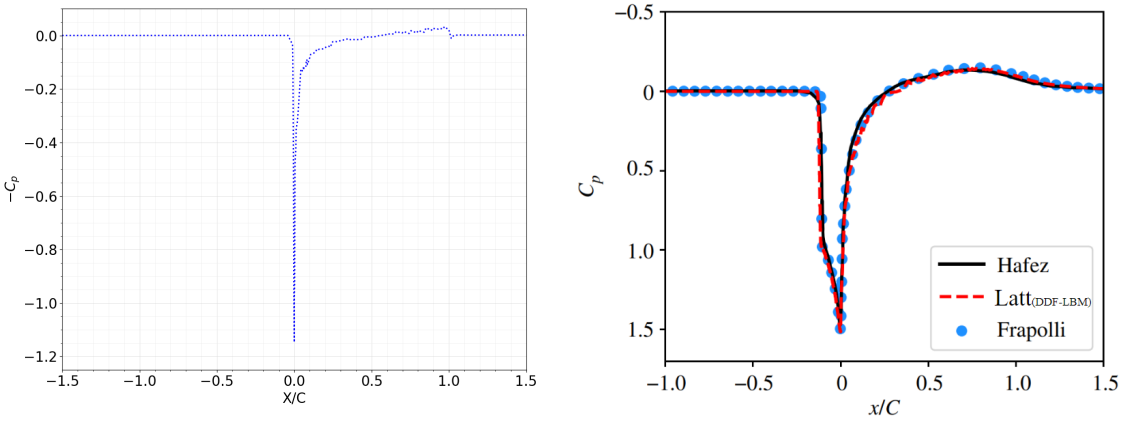
(a) Newton Raphson iterations (b) α variation

Figure 4.19: Number of Newton Raphson iterations and alpha variation around airfoil for Mach number = 1.4 and Re = 300

As in the previous benchmark, the number of iteration counts is around 3 – 5 iterations. The highest number of iterations can be observed near the leading edge of the airfoil as well as in the areas where vortex shedding is initiated. Further, the variation of α represents the applicability of the Knudsen number-dependent stabilization across the domain. Therefore, it is evident that stable simulations can be performed with a lower number of Newton-Raphson iterations for the presented benchmarking cases. However, apart from stability, it is also required to verify the accuracy of the simulation results. To investigate accuracy, the pressure coefficient around the airfoil is calculated according to equation 4.17 as mentioned in [10]. The simulated pressure coefficient is compared with the results taken from [6].

$$C_p = \frac{P - P_\infty}{0.5\rho_\infty u_\infty^2} \tag{4.17}$$

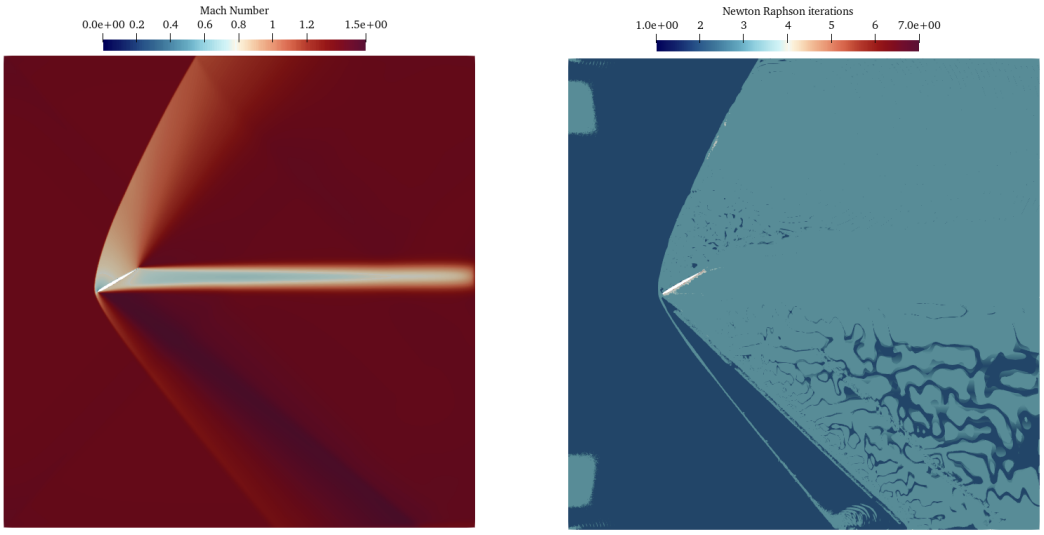
As shown in figure 4.20a, the pressure coefficient, C_p, is plotted against the distance normalized by the airfoil chord length. The range $x/c \in [-1.5, 0]$ corresponds to the upstream flow, while $x/c \in [-1.0, 1.5]$ corresponds to the downstream flow. The upper surface of the airfoil is within the normalized distance of $x/c \in [0, 1.0]$. Interestingly, even though the solver does not incorporate any grid refinement techniques, the variation in pressure coefficient closely follows the trends depicted in the literature, as shown in figure 4.20b. However, since the solver consists of standard lattice stencils, it cannot compete with the accuracy obtained by the higher-order lattice stencils used in previous compressible LBM approaches, such as in [6],[7].
Furthermore, the staircase approximation of the airfoil surface, resulting from the utilization of simple bounce-back scheme, may have also affected the results of the current implementation. Lastly, as a showcase simulation, an airfoil with a 30° angle is tested with a Mach number of 1.5 and a Reynolds number of 10^4. The Mach number profile and the number of Newton-Raphson iterations inside the domain are shown in figures 4.21a and 4.21b. Here, the effect of a strong bow shock can be observed on the upper surface of the airfoil.



(a) Variation of C_p with normalized distance according to current implementation

(b) Variation of C_p with normalized distance according to [6]

Figure 4.20: Pressure coefficient variation around airfoil uppersurface for Mach number = 1.5 and Re = 10^4



(a) Variation of Mach number profile around the airfoil with 30^0 angle

(b) Variation of Newton Raphson iterations inside the domain

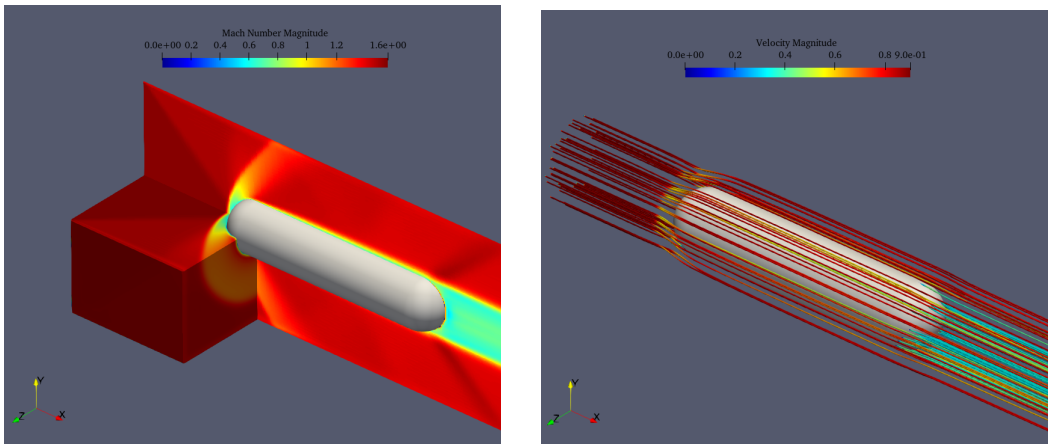
Figure 4.21: Mach number variation and number of Newton Raphson iterations around NACA0012 with 30^0 angle for Mach number = 1.5 and Re = 10^4

Until now, all the simulation benchmark cases have been performed considering two-dimensional scenarios. In the next section, an extension of the present method for three-dimensional domains will be showcased.

4.6 Compressible Flow Over Three-Dimensional Cylindrical Tube

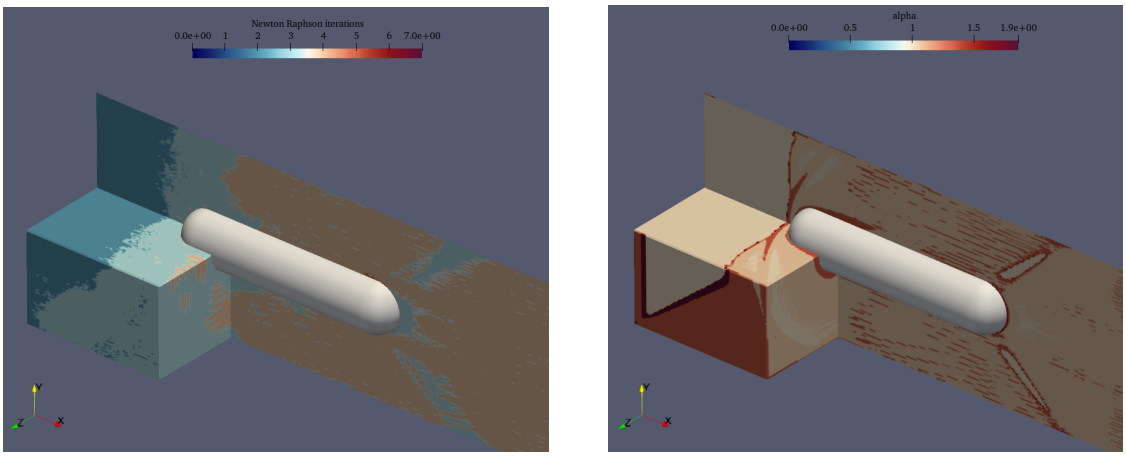
In this section, supersonic flow over a 3-D cylindrical tube is simulated using the present compressible LBM solver mentioned in Algorithm 3. For this case, the D3Q27 standard stencil is utilized. Apart from the difference in stencil configuration, there are several minor changes in the 3-D solver when estimating the pressure tensor and Newton-Raphson scheme. In this case, the pressure tensor is a 3x3 tensor, and tensor calculations should be properly adapted to this configuration. One of the key changes in the root-finding scheme is that now the number of Lagrangian multipliers is four, according to the constraint specified in equation 3.19. However, apart from these minor changes, the standard collide-and-stream algorithm remains the same, along with the other kernels.
The domain size is 1000x150x150 in the x, y, and z directions, respectively. The cylindrical tube is placed at the (102, 75, 75) coordinate inside the 3-D domain. The Mach number and Reynolds number for the present simulation are 1.4 and 3500, respectively. Apart from that, all other simulation parameters remain the same as in the previous section. Furthermore, the same boundary conditions for the inlet, outlet, top, and bottom boundaries are adopted. Since it is now 3-D, the front and back boundaries are also modeled using free stream boundary conditions. Moreover, similar to previous sections, the cylinder tube surface has a no-slip boundary condition according to the simple bounce-back scheme.
The simulated results for flow over the 3-D cylindrical case are shown in the following figures. Interestingly, the maximum number of Newton-Raphson iterations is in a similar range as in the 2-D case. Therefore, a stable simulation can be achieved with a relatively low number of iterations with the Knudsen number-dependent stabilization. As shown in figure 4.23a, the formation of a Mach cone near the front of the cylinder can be partly observed. Since

the discussed benchmarks show some prominent results, in the next chapter, the performance measurements of the present implementation is investigated.



(a) Mach number variation (b) Streamlines for velocity magnitude

Figure 4.22: Mach number variation and stream lines around cylindrical tube



(a) Number of Newton Raphson iterations (b) variation of alpha

Figure 4.23: Number of newton Raphson iterations and alpha function variation around cylindrical tube

Chapter 5

Performance Measurements

In this chapter, the performance evaluation of the implemented compressible LBM algorithm is discussed. In the first section, the performance measurements of the 2-D subsonic and supersonic algorithms are investigated. In the second section, the performance measurements related to the supersonic 3-D algorithm are discussed. Furthermore, both analyses are compared with the roofline performance model with respect to a relevant test system.

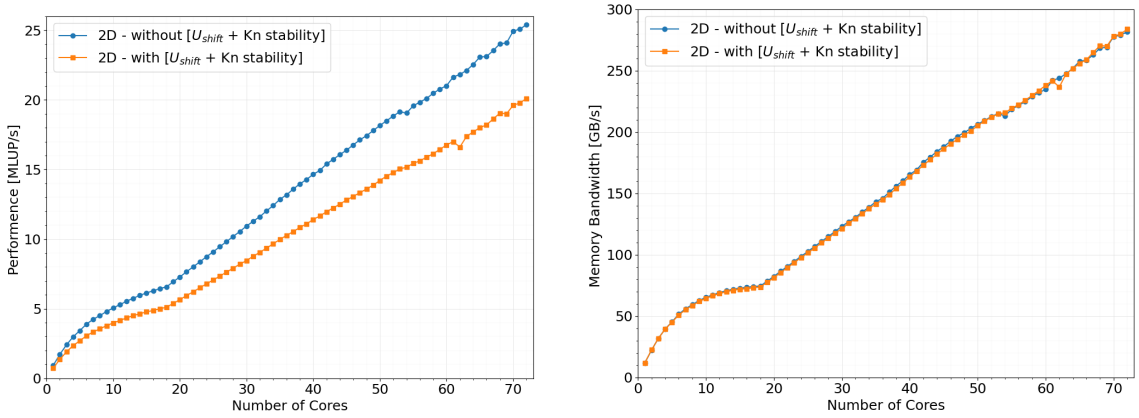
5.1 Performance Measurements of 2-D Solver

As mentioned in earlier sections, one of the primary advantages of using LBM is the ease of implementation and the excellent parallelizable capabilities of the method. Therefore, along with its accuracy and stability, it is vital to investigate its efficient implementation. Thus, the performance of the implemented compressible LBM is investigated to observe its scalability due to the additional kernels compared to standard LBM. To summarize the 2-D implementation details, the solver is implemented in C++, and all implementation kernels are parallelized using OpenMP. Furthermore, D2Q9 stencils are utilized with two discrete distribution functions representing each lattice node for mass, momentum, and energy conservations respectively.
All the benchmarkings are carried out on Friedrich-Alexander-Universität's (FAU) "Fritz" cluster, which is a high-performance computing resource with high-speed interconnect [32]. A concise description of a node topology of "Fritz" is provided in table 5.1. All the measurement data is obtained using LIKWID performance tools, which is a command line performance tool suite for the GNU/Linux operating systems [33].

	properties
processor	Intel Xeon Platinum 8360Y "IceLake" Processor
base frequency	2.4 GHz
total cores	72
sockets	2 (36 cores per socket)
NUMA domains	4
main memory	256 GB
single core last level cache	54 MB

Table 5.1: Specifications of a "Fritz" node: Simultaneous Multi-Threading (SMT) is disabled, and Cluster-on-Die (COD) is activated

The previously verified Sod shock tube case is subjected to the present benchmark analysis with a domain size of 3000x3000 lattice nodes in each x and y direction. One of the first steps of the analysis is to perform the strong scaling study. Strong scaling involves increasing the number of processors while keeping the amount of work constant [34]. To pin the OpenMP threads, the `omp_places` parameter is set to cores, which pins the threads to physical cores. In figure 5.1a, the performance is plotted against the number of cores, where Mega Lattice Updates per second (MLUP/s) is taken as the performance metric. In figure 5.1b, the memory bandwidth measurements against the number of cores can be observed.



(a) Performance vs number of cores

(b) Memory bandwidth vs number of cores

Figure 5.1: Strong scaling results (a) and memory bandwidth measurements (b) across a single node for 2-D

According to the observations, it can be seen that the performance and memory bandwidth saturate in the first NUMA domain in the first socket before scaling linearly. However, after that, the main algorithm seems to show excellent scaling trends. According to the literature, it is evident that LBM algorithms are generally memory-bound [1],[35]. Therefore, to investigate whether the implementation is compute-bound or memory-bound, a roofline analysis is performed. In a roofline model, the performance (Flops/s) is plotted against the operational intensity (Flops/Bytes) [36]. An empirical roofline model is constructed according to [37] using the capabilities of LIKWID tools. Rather than calculating the theoretical peak performance and theoretical memory bandwidth, respective measurements are also measured by using `likwid-bench -t peakflops_sse` and `likwid-bench -t load_sse`. Since, the `likwid-bench -t load_*` provides highest memory bandwidth, it is considered for the measurements [37]. Even though higher horizontal roofs can be obtained, the `likwid-bench -t peakflops_sse` is taken as the measurements to construct a realistic roofline model. Thereafter, the position of the compressible LBM algorithm is also investigated. According to the figure 5.2, it is evident that the present implementation is close to memory bound likewise the conventional LBM cases.

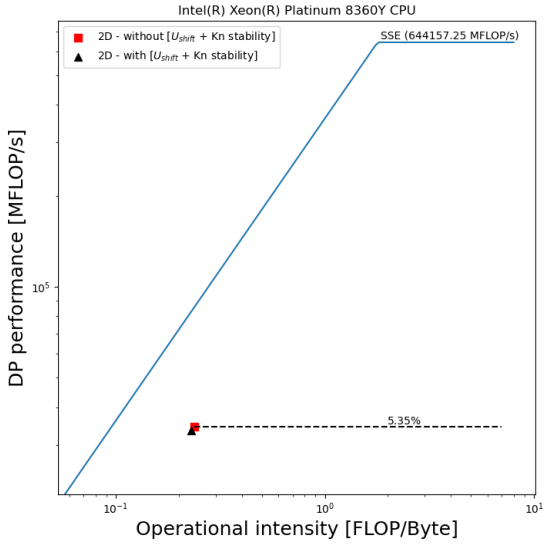
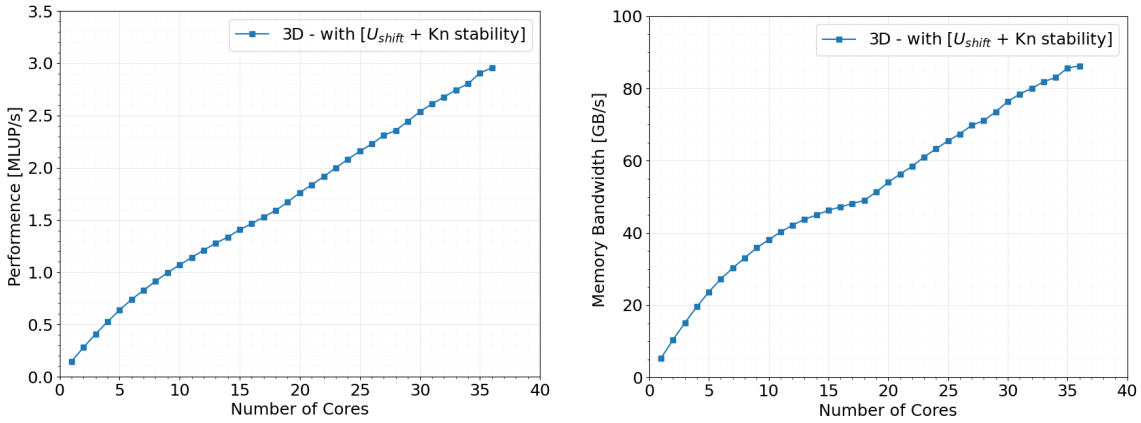


Figure 5.2: Roofline model for Intel(R) Xeon(R) Platinum 8360Y; "Icelake" CPU, along with the compressible LBM implementation for 2-D

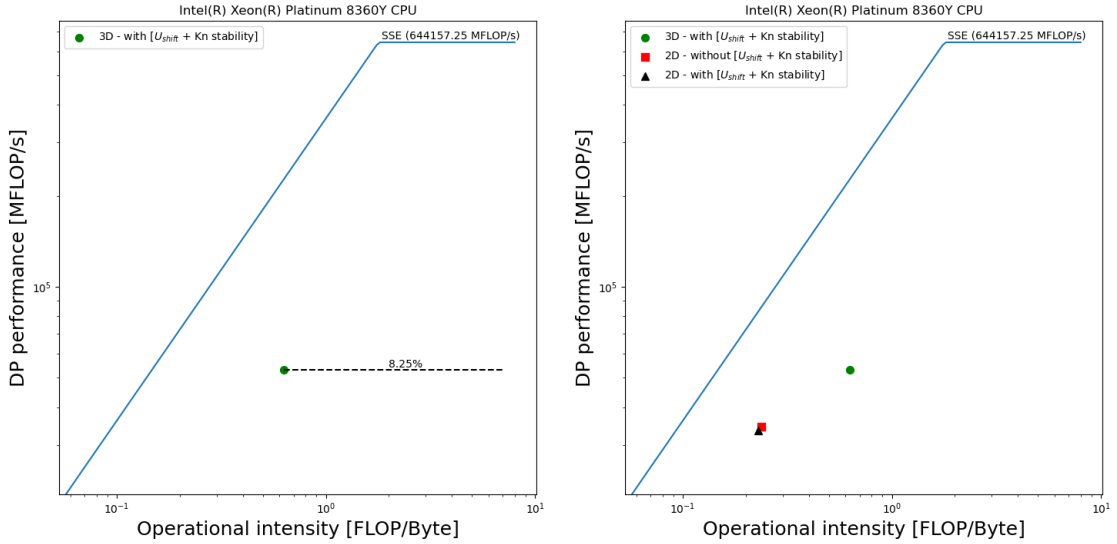
5.2 Performance Measurements of 3-D Solver

In this section, the performance measurements of the 3-D shock tube are investigated. The 3-D domain is consists of 220x220x220 lattice nodes in each x, y and z axes. The test system is also same as the previous case along with other technical settings. One of the main difference compared to the above 2-D case is the use of D3Q27 standard stencil. Figure 5.3a and 5.3b represent the preformance (MLUP/s) and the memory bandwidth measurements against the number of cores. As observed in the previous case, the memory saturation can be observed first NUMA domain and, it scales after 18 cores. However, a clear saturation cannot be observed in performance measurements in figure 5.3a. However, a slight saturation can be observed at 18 cores. Even though, increasing of domain size may helps to observe a clear performance saturation, the domain size is limited to around 220^3 with present data structures used in present compressible LBM implementation.
However, similar to the 2-D case, the 3-D compressible LBM solver marks another horizontal roof, with present roofline analysis as shown in figure 5.4a. Therefore, it is also evident that the present compressible LBM is close to memory bound. Therefore, maybe there are more opportunity to increase bandwidth utilization, for example by using SIMD instructions. In figure 5.4b, a comparison of both the 3-D and 2-D implementation is depicted.



(a) Performance vs number of cores (b) Memory bandwidth vs number of cores

Figure 5.3: Strong scaling results (a) and memory bandwidth measurements across the first socket (36 cores) for 3-D



(a) 3-D (b) 2-D and 3-D

Figure 5.4: Roofline model for Intel(R) Xeon(R) Platinum 8360Y; "Icelake" CPU, along with the compressible LBM implementation for 3-D (a) and comparison between 2-D and 3-D (b)

Chapter 6

Conclusion

In this concluding section, a summary of the key findings and contributions of the present research work is discussed along with their implications. Additionally, recommendations for future studies are outlined.

6.1 Concluding Remarks

The LBM is an emerging numerical technique in the area of CFD. Specifically, the simplicity and parallelizable capabilities of the LBM caught the eye of the CFD community, due to the fact that conventional CFD techniques required much more computational cost, even though they are accurate compared to the LBM. However, the standard LBM is widely used and investigated in the incompressible flow regime.
Furthermore, the standard LBM is limited to low Mach numbers. In contrast, in this research work, a compressible LBM solver aiming for flows with high Mach numbers and high Reynolds numbers has been developed and investigated, yielding several noteworthy findings. The present solver is based on the standard collide and stream algorithm, where the relaxation time is estimated by minimizing the discrete entropy function. Additionally, two sets of discrete distribution functions are utilized to represent mass, momentum, and energy conservation. Therefore, the solver falls into the category of entropic LBM (ELBM).
The developed compressible LBM solver can accurately solve the Sod shock tube case, which is a form of a classical Riemann problem. Therefore, the compressible solver is successfully validated and verified. Furthermore, the compressible LBM solver is also validated and verified according to canonical CFD benchmark cases such as flow over a 2-D cylinder and flow over an NACA0012 airfoil. In previous literature, the recorded maximum reachable Mach number was 1.8-1.9. However, in this research, the Mach number can reach up to 2.0, as depicted in figure 4.15. Another interesting concept encountered during the investigation is the shifted lattice scheme and Knudsen number-dependent stabilization. The utilization of these techniques is key to extending the compressibility limit up to supersonic. Furthermore, the solver's capabilities to simulate a 3-D cylindrical tube are also presented.
The present solver is based on C++ along with OpenMP to enhance parallelizability. Nevertheless, aligned with the standard collide and stream algorithm, the compressible LBM consists of additional subroutines compared to the standard LBM. Thus, the performance capabilities of the solver are studied to understand the scalability behavior of a parallel algorithm. According to the observations, the main algorithm showed excellent scalability considering 2-D and 3-D simulation domains. The maximum achievable performance for the algorithm considering a 2-D domain on an Intel(R) Xeon(R) Platinum 8360Y CPU @ 2.40 GHz with 72 Intel Icelake processor cores is approximately 24 MLUP/s, while in 3-D, it is approximately 5 MLUP/s.

Furthermore, according to the roofline performance model analysis, it can be seen that the present compressible algorithm is close to memory-bound, similar to conventional LBM codes. Since the main aim of the study is not about the performance analysis of the solver, the development of an MPI version or a GPU-capable solver is left for future works. Apart from that, additional recommendations for future work are discussed in the next section.

6.2 Future Research Work

Even though the present compressible LBM solver is successfully verified and validated, inviscid test cases, such as the isentropic vortex problem, demonstrated that the accuracy of vortical flows needs some improvements. However, since the expectations of the solver match with the research objectives, further improvements are left for future works. One suggested method to eliminate deviations of the fourth-order moment is to add correcting terms locally to g^{eq} and g^{*} according to the suggestion by Saadat et al. [8]. However, adding correction terms will incorporate additional finite difference schemes into the algorithm, deviating the solver from being an LB method. Thus, these kinds of schemes can be categorized as LB models rather than LB methods [16]. The effect of not having correction terms in the present compressible LBM solver can be seen in the following figure 6.1. In this isentropic vortex, the pressure contours should be stable throughout the simulation time. However, with the present solver, the pressure contours have some spurious oscillations. Therefore, one of the future studies is to implement the correction terms without deviating from the LB general workflow.

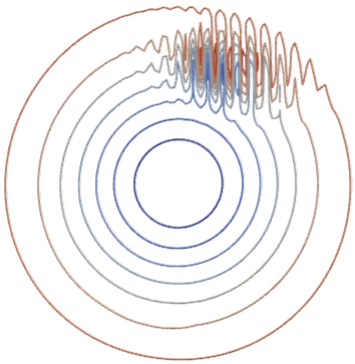


Figure 6.1: Spurious oscillations in pressure contours due to the lack of correction terms for a vortex advecting with Ma = 0.8

As mentioned before, improving the parallelizing capabilities of the solver is critical when performing large-scale simulations. However, rather than developing a perfect code from scratch, it is practical to use existing LBM frameworks for that purpose. waLBerla (widely applicable Lattice Boltzmann from Erlangen) is a state-of-the-art open-source software framework created especially to tackle performance issues and support intricate multiphysics simulations, utilizing the full potential of the greatest supercomputers [38]. However, the capabilities of waLBerla are mainly in the realm of incompressible fluids when it comes to fluid simulations. Therefore, one of the main future works is to integrate the present solver into the waLBerla framework.
Moreover, it will be interesting to simulate Fluid-Structure-Interaction (FSI) simulations along with the compressible LBM solver. Therefore, it is also one of the future works to develop a

coupled compressible LBM solver with discrete element method (DEM) or discrete practical simulations (DPS) to perform coupled fluid simulations.
As we revisit the initial objectives outlined in the introduction, it is evident that this research has effectively addressed and achieved the stated objectives. The alignment between our research questions and the obtained results underscores the coherence of the study. In conclusion, this thesis has presented a comprehensive exploration of compressible LBM. While the study has made significant strides in compressible LBM, there remains an exciting landscape for future researchers to explore. The knowledge generated through this research serves as a foundation for further advancements in compressible LBM. We hope that the findings presented here inspire continued inquiry and contribute to the ongoing pursuit of knowledge in Computational Science and Engineering (CSE).

Bibliography

[1] Timm Krüger, Halim Kusumaatmaja, Alexandr Kuzmin, Orest Shardt, Goncalo Silva, and Erlend Magnus Viggen. *The Lattice Boltzmann Method*. Springer International Publishing, 2017.

[2] Chenghai Sun. Lattice-boltzmann models for high speed flows. *Physical Review E*, 58(6):7283–7287, dec 1998.

[3] Ho-Keun Kang, Michihisa Tsutahara, Ki-Deok Ro, and Young-Ho Lee. Numerical simulation of shock wave propagation using the finite difference lattice boltzmann method. *KSME International Journal*, 16(10):1327–1335, oct 2002.

[4] Takeshi Kataoka and Michihisa Tsutahara. Lattice boltzmann method for the compressible euler equations. *Physical Review E*, 69(5):056702, may 2004.

[5] N. Frapolli, S. S. Chikatamarla, and I. V. Karlin. Entropic lattice boltzmann model for gas dynamics: Theory, boundary conditions, and implementation. *Physical Review E*, 93(6):063302, jun 2016.

[6] Jonas Latt, Christophe Coreixas, Joël Beny, and Andrea Parmigiani. Efficient supersonic flow simulations using lattice boltzmann methods based on numerical equilibria. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 378(2175):20190559, jun 2020.

[7] Frapolli, Nicolò. *Entropic lattice Boltzmann models for thermal and compressible flows*. PhD thesis, 2017.

[8] Mohammad Hossein Saadat, Fabian Bösch, and Ilya V. Karlin. Lattice boltzmann model for compressible flows on standard lattices: Variable prandtl number and adiabatic exponent. *Physical Review E*, 99(1):013306, jan 2019.

[9] M. H. Saadat, S. A. Hosseini, B. Dorschner, and I. V. Karlin. Extended lattice boltzmann model for gas dynamics. *Physics of Fluids*, 33(4), apr 2021.

[10] Si Bui Quang Tran, Fong Yew Leong, Quang Tuyen Le, and Duc Vinh Le. Lattice boltzmann method for high reynolds number compressible flow. *Computers and Fluids*, 249:105701, dec 2022.

[11] Michel A. Saad. *Compressible Fluid Flow (2nd Edition)*. Prentice Hall, 1998.

[12] A. A. Mohamad. *Lattice Boltzmann Method Fundamentals and Engineering Applications with Computer Codes*. Springer, 2019.

[13] Martin Bauer, Harald Köstler, and Ulrich Rüde. lbmpy: Automatic code generation for efficient parallel lattice boltzmann methods. *Journal of Computational Science*, 49:101269, feb 2021.

[14] Nikolaos I. Prasianakis and Iliya V. Karlin. Lattice boltzmann method for thermal flow simulation on standard lattices. *Physical Review E*, 76(1):016702, jul 2007.

[15] P. L. Bhatnagar, E. P. Gross, and M. Krook. A model for collision processes in gases. i. small amplitude processes in charged and neutral one-component systems. *Physical Review*, 94(3):511–525, may 1954.

[16] Renard,Florian. *Hybrid Lattice Boltzmann Method for CompressibleFlows*. PhD thesis, 2021.

[17] Christophe Coreixas and Jonas Latt. Compressible lattice boltzmann methods with adaptive velocity stencils: An interpolation-free formulation. *Physics of Fluids*, 32(11), nov 2020.

[18] Bruce M Boghosian, Jeffrey Yepez, Peter V Coveney, and Alexander Wager. Entropic lattice boltzmann methods. *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences*, 457(2007):717–766, mar 2001.

[19] KUN QU, CHANG SHU, and YONG TIAN CHEW. Simulation of shock-wave propagation with finite volume lattice boltzmann method. *International Journal of Modern Physics C*, 18(04):447–454, apr 2007.

[20] Zhaoli Guo, Chuguang Zheng, Baochang Shi, and T. S. Zhao. Thermal lattice boltzmann equation for low mach number flows: Decoupling model. *Physical Review E*, 75(3):036704, mar 2007.

[21] N. Frapolli, S. S. Chikatamarla, and I. V. Karlin. Entropic lattice boltzmann model for compressible flows. *Physical Review E*, 92(6):061301, dec 2015.

[22] Ilya Karlin and Pietro Asinari. Factorization symmetry in the lattice boltzmann method. *Physica A: Statistical Mechanics and its Applications*, 389(8):1530–1548, apr 2010.

[23] S. P. Venkateshan and Prasanna Swaminathan. *Computational Methods in Engineering*. Academic Press.

[24] Robert A. Meyers. *Encyclopedia of Physical Science and Technology, 3rd Edition, 18 volume set*. Academic Press, 2001.

[25] Eleuterio F. Toro. *Riemann Solvers and Numerical Methods for Fluid Dynamics*. Springer.

[26] Bruce Fryxell. Cococubed.com. https://cococubed.com/code_pages/exact_riemann.shtml, cited November 2023.

[27] Yousif Badri, Aboubaker Elbashir, Ahmad Saker, and Samer F. Ahmed. An experimental and numerical investigation of the effects of the diaphragm pressure ratio and its position on a heated shock tube performance. *Energy Science and Engineering*, 10(4):1177–1188, feb 2022.

[28] Gary A Sod. A survey of several finite difference methods for systems of nonlinear hyperbolic conservation laws. *Journal of Computational Physics*, 27(1):1–31, apr 1978.

[29] N. Frapolli, S. S. Chikatamarla, and I. V. Karlin. Lattice kinetic theory in a comoving galilean reference frame. *Physical Review Letters*, 117(1):010604, jun 2016.

[30] Gang Mei, Liangliang Xu, and Nengxiong Xu. Accelerating adaptive inverse distance weighting interpolation algorithm on a graphics processing unit. *Royal Society Open Science*, 4(9):170436, sep 2017.

[31] Vincente Cardona and Viviana Lago. Generalized shock stand-off distance equation of a sphere with dependence in viscosity. *Physics Letters A*, 491:129185, December 2023.

[32] NHRFAU. Fritz parallel cluster (nhr+tier3). https://hpc.fau.de/systems-services/documentation-instructions/clusters/fritz-cluster/, cited January 2024.

[33] Jan Treibig, Georg Hager, and Gerhard Wellein. Likwid: A lightweight performance-oriented tool suite for x86 multicore environments. In *2010 39th International Conference on Parallel Processing Workshops*. IEEE, September 2010.

[34] HPCWIKI. Scaling. https://hpc-wiki.info/hpc/Scaling, cited January 2024.

[35] M. Wittmann, V. Haag, T. Zeiser, H. Köstler, and G. Wellein. Lattice boltzmann benchmark kernels as a testbed for performance analysis. *Computers and Fluids*, 172:582–592, August 2018.

[36] João Manuel Paiva Cardoso, José Gabriel de Figueired Coutinho, and Pedro C. Diniz. *Embedded Computing for High Performance*. Elsevier Science and Technology Books, 2017.

[37] Thomas Gruber. Empirical roofline model. https://github.com/RRZE-HPC/likwid/wiki/Tutorial%3A-Empirical-Roofline-Model, cited January 2024.

[38] Martin Bauer, Sebastian Eibl, Christian Godenschwager, Nils Kohl, Michael Kuron, Christoph Rettinger, Florian Schornbaum, Christoph Schwarzmeier, Dominik Thönnes, Harald Köstler, and Ulrich Rüde. walberla: A block-structured high-performance framework for multiphysics simulations. *Computers and Mathematics with Applications*, 81:478–501, January 2021.

Appendix A

Derivations

A.1 Derivation of discrete equilibrium distribution using Lagrangian Multipliers

$$\text{Minimize} : H = \sum_{i=0}^{Q-1} g_i \cdot ln\left(\frac{g_i}{\kappa}\right) \tag{A.1}$$

subjected to:

$$\text{Energy} : \sum_{i=0}^{Q-1} g_i = 2\rho E \tag{A.2}$$

$$\text{Trace of third order Maxwell-Boltzmann moment} : q_\alpha^{eq} = \sum_{i=0}^{Q-1} c_{i\alpha} g_i^{eq} = 2\rho u_\alpha (E+T) \tag{A.3}$$

Let's consider a 2d case and restate the constraints accordingly,

$$\sum_{i=0}^{Q-1} g_i = 2\rho E \tag{A.4}$$

$$\sum_{i=0}^{Q-1} c_{ix} g_i^{eq} = 2\rho u_x (E+T) \tag{A.5}$$

$$\sum_{i=0}^{Q-1} c_{iy} g_i^{eq} = 2\rho u_y (E+T) \tag{A.6}$$

By constructing the Lagrangian function and taking the derivative with respect to discrete distribution function considering g_i,

$$L = \sum_{i=0}^{Q-1} g_i \cdot ln\left(\frac{g_i}{\kappa}\right) + \lambda\left(\sum_{i=0}^{Q-1} g_i - 2\rho E\right) + \mu_1\left(\sum_{i=0}^{Q-1} c_{ix} g_i^{eq} - 2\rho u_x (E+T)\right) + \mu_2\left(\sum_{i=0}^{Q-1} c_{iy} g_i^{eq} - 2\rho u_y (E+T)\right) \tag{A.7}$$

$$\Delta L = 1 + ln\left(\frac{g}{\kappa}\right) + \lambda + \mu_1(c_x) + \mu_2(c_y) \tag{A.8}$$

Equating A.8 to zero the minimum g can be obtained as follows;

$$g = \kappa exp^{(-1-\lambda+\mu_1(c_x)+\mu_2(c_y))} \tag{A.9}$$

Finally, the general form of equilibrium distribution function g_i^{eq} can be stated as follows using the Einstein index notataion,

$$g_i^{eq} = \kappa \; exp(-1(1 + \lambda + \mu_\alpha c_{i\alpha})) \tag{A.10}$$

A.2 f_i^{eq} considering D2Q9 stencil

$$f_i^{eq} = \rho \; \phi_{ix} \cdot \phi_{iy} \cdot \phi_{iz} \tag{A.11}$$

where,

$$\phi_{(0)} = 1 - ((u_\alpha - U_\alpha)^2 + T) \tag{A.12}$$

$$\phi_{(+1)} = \frac{(u_\alpha - U_\alpha) + (u_\alpha - U_\alpha)^2 + T}{2} \tag{A.13}$$

$$\phi_{(-1)} = \frac{-(u_\alpha - U_\alpha) + (u_\alpha - U_\alpha)^2 + T}{2} \tag{A.14}$$

$$\begin{aligned}
f_0^{eq} &= \rho \; \phi_0 \cdot \phi_0 \\
f_1^{eq} &= \rho \; \phi_{+1} \cdot \phi_0 \\
f_2^{eq} &= \rho \; \phi_0 \cdot \phi_{+1} \\
f_3^{eq} &= \rho \; \phi_{-1} \cdot \phi_0 \\
f_4^{eq} &= \rho \; \phi_0 \cdot \phi_{-1} \\
f_5^{eq} &= \rho \; \phi_{+1} \cdot \phi_{+1} \\
f_6^{eq} &= \rho \; \phi_{-1} \cdot \phi_{+1} \\
f_7^{eq} &= \rho \; \phi_{-1} \cdot \phi_{-1} \\
f_8^{eq} &= \rho \; \phi_{+1} \cdot \phi_{-1}
\end{aligned}$$

$$\begin{aligned}
f_0^{eq} &= \rho(1 - ((u_x - U_x)^2 + T))(1 - ((u_y - U_y)^2 + T)) \\
f_1^{eq} &= \rho(\frac{(u_x - U_x) + (u_x - U_x)^2 + T}{2})(1 - ((u_y - U_y)^2 + T)) \\
f_2^{eq} &= \rho(1 - ((u_x - U_x)^2 + T))(\frac{(u_y - U_y) + (u_y - U_y)^2 + T}{2}) \\
f_3^{eq} &= \rho(\frac{-(u_x - U_x) + (u_x - U_x)^2 + T}{2})(1 - ((u_y - U_y)^2 + T)) \\
f_4^{eq} &= \rho(1 - ((u_x - U_x)^2 + T))(\frac{-(u_y - U_y) + (u_y - U_y)^2 + T}{2}) \\
f_5^{eq} &= \rho(\frac{(u_x - U_x) + (u_x - U_x)^2 + T}{2})(\frac{(u_y - U_y) + (u_y - U_y)^2 + T}{2}) \\
f_6^{eq} &= \rho(\frac{-(u_x - U_x) + (u_x - U_x)^2 + T}{2})(\frac{(u_y - U_y) + (u_y - U_y)^2 + T}{2}) \\
f_7^{eq} &= \rho(\frac{-(u_x - U_x) + (u_x - U_x)^2 + T}{2})(\frac{-(u_y - U_y) + (u_y - U_y)^2 + T}{2}) \\
f_8^{eq} &= \rho(\frac{(u_x - U_x) + (u_x - U_x)^2 + T}{2})(\frac{-(u_y - U_y) + (u_y - U_y)^2 + T}{2})
\end{aligned}$$

Appendix B

Algorithms

B.1 grid class skeleton

Algorithm 4: Implementation of grid class.

```
1  class grid{
2  public:
3      /* Number of elements in spatial coordinates */
4      int NX;
5      int NY;
6      int NZ;
7      /* Pointer to a node object */
8      node* domain;
9      /* Constrctor */
10     grid();
11     /* Constrctor */
12     grid(int nx, int ny, int nz);
13     /* Copy constrctor */
14     grid(const grid& src);
15     /* Copy assignment operator */
16     grid& operator =(const grid& src);
17     /* Overload operator() */
18     node &operator()(int k, int j, int i);
19     /* Destructors */
20     ~grid();
21 };
```

B.2 GSL inverse finder

Algorithm 5: GSL implementation for finding inverse of a matrix.

```
1  void  inverse(int n, double* INV, double* j){
2        gsl_matrix_view m = gsl_matrix_view_array(j, n, n);
3        double temp[n*n];
4        gsl_matrix_view inv = gsl_matrix_view_array(temp,n,n);
5        gsl_permutation* p = gsl_permutation_alloc(n);
6        gsl_linalg_LU_decomp(&m.matrix, p, &s);
7        gsl_linalg_LU_invert (&m.matrix, p, &inv.matrix);
8  for (int i = 0; i < n; ++i)
9  {
10 |   for (int j= 0; j < n; ++j)
11 |   {
12 |   |   INV[n*i+j] = gsl_matrix_get(&inv.matrix,i,j);
13 |   }
14 }
15 gsl_permutation_free(p);
16 };
```